

UNIT STUDY NOTES

Pythonic Thinking: Iterators, Copying and Identity

Idiomatic Python is less about clever syntax and more about the object model underneath it. This unit covers the iterator protocol, the functional builtins, shallow versus deep copying, mutability versus immutability, and how arguments are actually bound.

11 source notebooks

112 topic sections extracted

9 study sections

Unit Overview

In the previous notebook we built an iterator the hard way: one class to hold the data, another class to walk it, `__iter__` and `__next__` methods, a manual cursor and an explicit `StopIteration`. It worked — and it was tedious. Generators are Python's answer to that tedium: a **generator** is a special kind of iterator you get almost for free by writing a normal function and using the `yield` keyword instead of `return`.

Why does this matter beyond saving keystrokes? Because generators make *lazy computation* the default. A generator never builds a whole result in memory; it produces one value, pauses, waits for you to ask again, then resumes exactly where it left off — like a vending machine that dispenses one snack at a time instead of dumping the entire inventory on the floor. That single property is what lets Python programs loop over a million files, a live sensor feed, or a huge dataset without running out of RAM.

Here is the roadmap for this lesson:

1. Recall the manual iterable/iterator class pair (`mera_range` + `mera_iterator`) that generators replace.
2. Write our first generator function and observe that calling it runs **no** code until the first `next()` or `for` pull.
3. Compare `yield` with `return`: pausing versus finishing, resumable state versus one-shot results.
4. See how generators remember their position between pulls, and how they die once exhausted.
5. Rebuild `range()` as a six-line generator — `mera_range` again, but with `yield`.
6. Use **generator expressions**, the lazy twin of list comprehensions.
7. Study a practical lazy reader that yields images one at a time, then walk through the four headline benefits: easier implementation, tiny memory, infinite streams, and chained pipelines.

Key definitions. A **generator function** is any function containing `yield`; calling it returns a **generator object**, which is a full iterator. Each `next()` runs the body until the next `yield`, whose value is sent back; when the function falls off the end, the generator raises `StopIteration`. Hold onto those five sentences — the rest of the notebook is just variations on them.

Loops look simple on the surface: `for i in num:` and Python hands you one item at a time. Underneath that single line sits a tiny, elegant contract between two kinds of objects — the **iterable**, which owns the data, and the **iterator**, which knows how to walk through it. This notebook is about that contract. Once you can see it, a

surprising amount of Python (loops, generators, file reading, `zip`, `enumerate`, streaming APIs) stops being magic and becomes the same five lines of protocol repeated everywhere.

Why should you care? Because understanding iterators changes how you judge memory. A list of a hundred million integers can crash your program, while `range(100000000)` sits in memory comfortably — not because Python is clever about lists, but because `range` is an *iterator-style* object that computes each number only when you ask for it. The same idea powers generators, data pipelines, and lazy loading of files or images.

Here is the roadmap for this lesson:

1. Define **iteration**, **iterables** and **iterators**, and see that `iter()` and `next()` are the only two functions you really need.
2. Learn the quick `dir()` trick for telling an iterable from an iterator by looking for `__iter__` and `__next__`.
3. Unpack what a `for` loop actually does — spoiler: it calls `iter()` once, then keeps calling `next()` until `StopIteration` is raised.
4. Rebuild the loop ourselves as `mera_khudka_for_loop`, using `try` / `except StopIteration`.
5. Build our own `range()` from two classes, `mera_range` (the iterable) and `mera_range_iterator` (the iterator), so the protocol stops being abstract.

Keep three definitions in your back pocket. An **iterable** is any object whose `__iter__` method returns an iterator — lists, strings, dicts, sets, `range` objects and files all qualify. An **iterator** is an object with both `__iter__` (returning itself) and `__next__` (returning the next value or raising `StopIteration`). **Iteration** is simply the act of consuming an iterator until it is exhausted. Get those straight and this whole notebook is just variations on them.

Every name in a Python program — every variable, function, class, module — lives in a **namespace**: an object that maps names to the things they refer to. At module level you can think of a namespace as a dictionary; type `globals()` and you will literally see one. This notebook is about two related superpowers. First, *where Python looks* when you use a name — the **LEGB rule**, which decides whether `a` means your local variable, an enclosing function's variable, the module-level variable, or a built-in. Second, *what you can do with functions once you treat them as data* — passing them around, wrapping them, and finally the **decorator** syntax that lets you bolt extra behaviour onto a function without touching its body.

Why care? Because nearly every "why is my variable not updating?!" bug is a namespace bug: a function that assigns to a name silently creates a *local* variable instead of touching the global one. And nearly every framework you will meet — Flask, Django, pytest, click, FastAPI — is built on decorators, so reading `@app.route` or `@app.get` should feel ordinary, not mysterious.

The roadmap for this notebook:

1. **Namespaces and the LEGB rule** — the four scopes (built-in, global, enclosing, local), searched inside-out, with `NameError` as the final outcome.
2. **Local vs global in practice** — shadowing, read-only fallback, assignment surprises, and the `global` keyword for deliberate edits.
3. **The built-in scope** — the `builtins` module, and why redefining `max` (or `print`) is legal but dangerous.

4. **Enclosing scope and `nonlocal`** — nested functions reaching outward, and how `nonlocal` differs from `global`.
5. **Functions as first-class objects** — the prerequisite that makes decorators possible at all.
6. **Decorators** — the wrapper pattern by hand, the `@` syntax as shorthand, a genuinely useful `@timer`, the classic pitfalls, and finally **decorators with arguments** built from three nested functions.

Two definitions to carry forward. A **scope** is the region where a name is directly visible; a **closure** is a function plus the enclosing variables it remembers; a **decorator** is a callable that takes a function and returns a (usually enhanced) function. By the end, both halves of the notebook — name lookup and function wrapping — should feel like two views of the same idea: Python lets you intercept things, names and calls alike, at well-defined choke points.

Two small functions, `zip()` and `enumerate()`, remove a huge amount of everyday Python pain. `zip()` glues several sequences together element by element, so you never have to index them in parallel. `enumerate()` gives you the position of each item while you loop, so you never have to write `range(len(...))` again.

The big picture. Real data arrives in parallel columns: names next to marks, product names next to prices, ages next to cities. `zip()` interleaves those columns into tuples, and `enumerate()` stamps each row with a running index. Together they turn clumsy index arithmetic into readable, Pythonic code.

Why it matters. Both return **lazy iterators**, so they inherit the single-use behaviour from the iterators notebook. They also power common idioms: `dict(zip(keys, values))` to build a dictionary, `zip(*rows)` to transpose a table, and `sorted(zip(values, labels), reverse=True)` to rank items.

Roadmap. In this notebook you will:

1. Pair two or more lists with `zip()` and see its lazy, single-use nature.
2. Handle uneven inputs, including the `strict=True` guard.
3. Use the unzip trick `zip(*zipped)` and the `dict(zip(...))` factory.
4. Add indices with `enumerate()`, including a custom `start=`.
5. Replace `range(len(...))` and hunt positions cleanly.
6. Build a student leaderboard from a real CSV.

Key definitions. `zip(a, b)`: a lazy iterator of tuples `(a_i, b_i)` that stops at the shortest input. `enumerate(seq, start=0)`: a lazy iterator of `(index, item)` pairs. *Unzip*: `zip(*pairs)` separates a list of tuples back into parallel tuples.

Prerequisites

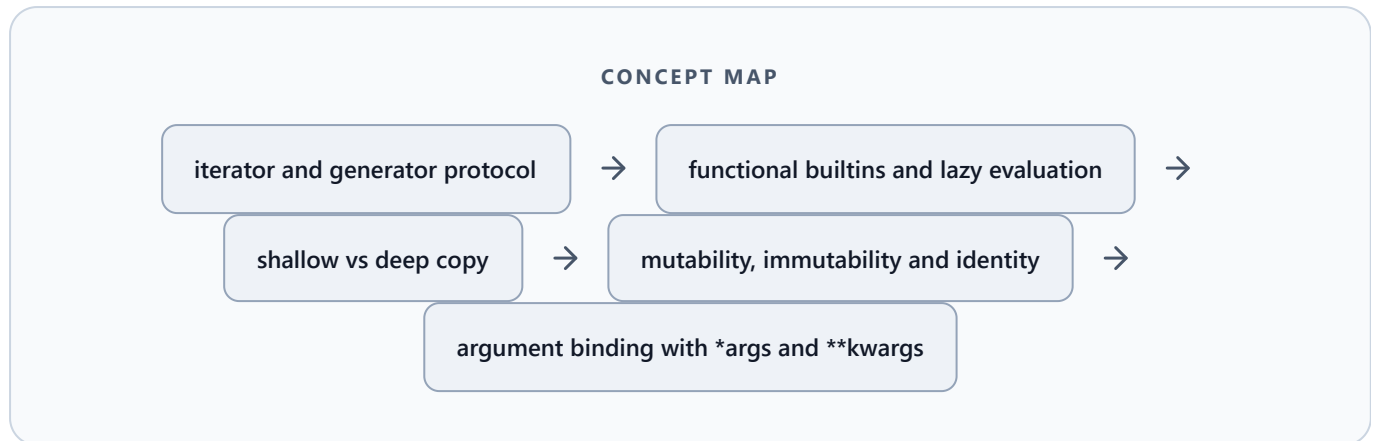
→ Built-in Data Structures and Functions

What you will be able to do

- ✓ Explain the iterator protocol and write generators that are lazy, not eager
- ✓ Compose map, filter, zip and reduce for readable data transformations
- ✓ Distinguish shallow, deep and structural copies and use copy/deepcopy correctly

✓ Bind arguments correctly with `*args`, `**kwargs` and keyword-only parameters

What This Unit Covers



The sections below follow this arc, from foundational semantics to the engineering decisions that follow from them.

Core Concepts

Generators — Pythonic Thinking

1. The boilerplate generators replace

1.1 Two classes, five methods — and then there was `yield`

Before writing a single generator, look hard at the code in the next cell, because it is the *before* picture of this entire lesson. `mera_range` is the iterable: it stores `start` and `end`, and its `__iter__` method hands back a `mera_iterator`. `mera_iterator` is the walker: it holds a reference back to the iterable, returns itself from `__iter__`, and in `__next__` either raises `StopIteration` (when `start >= end`) or returns the current number and nudges the cursor forward.

Count the moving parts: two classes, five methods, one piece of shared mutable state, and an explicit end-of-sequence check. Multiply that by every lazy data source you will ever write — file readers, database cursors, API paginators — and you begin to feel the weight.

This is the gap generators close. A generator function keeps the *logic* (`for i in range(start, end): yield i`) and deletes all the scaffolding: no classes, no `__next__`, no manual `StopIteration` — Python raises it for you the moment the function returns.

Keep this class version visible as a reference point. Whenever a generator example looks almost too simple to be correct, flip back to this cell and ask: which of these method bodies is Python now writing for me?

PYTHON

```

# iterable
class mera_range:
    def __init__(self, start, end):
        self.start = start
        self.end = end

    def __iter__(self):
        return mera_iterator(self)

# iterator
class mera_iterator:
    def __init__(self, iterable_obj):
        self.iterable = iterable_obj

    def __iter__(self):
        return self

    def __next__(self):
        if self.iterable.start >= self.iterable.end:
            raise StopIteration

        current = self.iterable.start
        self.iterable.start += 1
        return current

```

1.2 The why: values on demand

The next cell repeats a memory experiment you met in the iterators notebook, and the gap it exposes is enormous. The list comprehension `[x for x in range(100000)]` materialises every element immediately: each Python `int` is a full object, so the list alone weighs hundreds of kilobytes — the output of `sys.getsizeof(L)` makes that concrete in the benefits section later, where the same construction measures about 800 KB. `range(10000000)` — ten million numbers — is a different animal: it costs a fixed handful of bytes (the stored output here shows `48`), because it stores only a start, a stop and a step, computing element k only when someone asks for it.

Generators inherit exactly this philosophy, with one upgrade: they can be *arbitrary computations*, not just number sequences. Reading ten thousand CSV files? Yield one parsed row at a time. Training a model over images on disk? Yield one image array at a time. The producing code looks like a normal function with `yield`, but the consumption pattern stays lazy — pull, process, pull again.

Rule of thumb: **build a list when you genuinely need all the values at once; build a generator when you process values one by one.** Most loops process-and-discard, which is why most loops could quietly become generators.

Semantics

Generators preserve state between yields (coroutines-like pausing). Memory efficient, lazy.

Why

Generate values on-the-fly, avoid building large lists.

Use cases

Streaming, pipelines, large datasets.

Common mistakes

Mixing return/yield semantics.

PYTHON

```
L = [x for x in range(100000)]

#for i in L:
#    print(i**2)

import sys
sys.getsizeof(L)

x = range(10000000)

#for i in x:
#    print(i**2)
sys.getsizeof(x)
```

2. Your first generator

2.1 `yield`: a function that hands back values without finishing

The function in the next cell, `gen_demo`, looks like any other function except for one word: `yield`. Three `yield` statements replace three `return` statements — and that one word changes the function's entire nature.

Two-phase behaviour to internalise:

1. **Calling phase.** `gen = gen_demo()` runs *nothing* inside the body. No string is produced yet. What you get back is a **generator object** — a fresh iterator sitting at the top of the function, ready to execute.
2. **Production phase.** Only when something pulls — `for i in gen`, `next(gen)`, `sum(...)` — does Python run the body up to the first `yield`, pause it, hand the yielded value out, and remember *exactly* where it stopped: line number, local variables, the works. The next pull resumes on the line after that `yield`.

So iterating `gen` prints, one pull at a time, `first statement`, `second statement`, `third statement`. After the third, no `yield` remains; control falls off the end, the generator raises `StopIteration`, and the `for` loop ends.

Think of a generator as a ticket counter that rips one ticket per customer: it does not build the audience in advance, it serves on demand. Each `yield` is one ticket handed over; each pause is the counter waiting for the next customer.

One subtle consequence: code placed *before* the first `yield` still runs on the first pull, not at call time — a fact that trips people up when they put prints or setup logic at the top of a generator.

PYTHON

```
def gen_demo():  
    yield "first statement"  
    yield "second statement"  
    yield "third statement"
```

PYTHON

```
gen = gen_demo()  
  
for i in gen:  
    print(i)
```

2.2 `yield` versus `return`

Both keywords hand a value back to the caller, but they sit at opposite ends of the lifecycle:

	RETURN	YIELD
Effect on the function	Finishes it — the frame is destroyed	Pauses it — the frame stays alive behind the scenes
How many times per call	Exactly once	Once per <code>yield</code> , potentially forever
Caller gets	The value, then the function is gone	One value per pull; the function resumes later
End of output	Control returns immediately	<code>StopIteration</code> once no <code>yield</code> is left
What is remembered	Nothing after return	Locals, loop position, line number

A useful analogy: `return` is *mailing* a letter — you write it, send it, and it leaves your desk for good. `yield` is *bookmarking* a book and closing it — the book (your function's state) stays on the shelf exactly where you left off, ready for the next reader, again and again.

The empty cell below is a placeholder in this notebook for a Python Tutor visualisation. Watch, when you try it yourself, how a generator's frame reappears and disappears between pulls while a normal function's frame vanishes forever after a single `return`. And reread the `square` example later in this notebook with that picture in mind: it contains a `for` loop and a `yield`, so a single call builds no list of squares — each `next()` advances the loop by one step and freezes it again. A `return i**2` in the same place would return only the first square and then terminate.

2.3 Generators remember where they stopped

The next two cells are the clearest proof that a generator is a *stateful* iterator, not a function you re-run from the top.

`square(10)` is a generator function: calling it returns a generator without computing a single square. Then the notebook pulls three values explicitly:

Each call runs the loop body forward by one iteration and freezes it again. Crucially, the `for i in range(1, num + 1)` inside the generator did **not** restart — the generator remembers its loop variable between pulls. When the later `for i in gen:` loop runs, it continues with `16, 25, 36 ... 100`, which is exactly what the stored output shows: `1, 4, 9` from the explicit pulls, then `16` through `100` from the loop. One object, one continuous walk — the same behaviour as the `list_iterator` from the iterators notebook, expressed in ten lines of plain function code.

Two lifecycle rules follow:

- **Generators are single-pass.** Once the body finishes, the generator is exhausted; a further `next()` raises `StopIteration`. To walk the data again, call `square(10)` a second time to build a fresh generator.
- **Generators are cold until pulled.** Nothing inside the function body has executed at call time — you can even create a generator and never consume it, and it will cost almost nothing.

PYTHON

```
def square(num):
    for i in range(1,num+1):
        yield i**2
```

PYTHON

```
gen = square(10)

print(next(gen))
print(next(gen))
print(next(gen))

for i in gen:
    print(i)
```

3. `range()` again — now in four lines

3.1 The class pair collapses into one function

Remember `mera_range` from the iterators notebook: two classes, five methods, a manual `StopIteration` and a shared cursor? The next cell rebuilds the same idea as a generator:

That is the entire implementation. Walk through what Python now handles for you:

- Calling `mera_range(15, 26)` returns a generator — which is already an iterator, so the function object itself plays the role of the iterable.

- The loop variable `i` replaces the hand-written cursor; the built-in `range` does the counting.
- When the loop finishes, falling off the end of the function raises `StopIteration` automatically — no explicit check, no handwritten exception.
- Each `yield` freezes `i`'s value *and* the loop's progress, so consecutive pulls produce `15`, `16`, `17`, ... exactly like the class version.

The demo cell then loops over `mera_range(15, 26)` and prints `15` through `25` — the same stream the class version would produce, in roughly one-tenth of the code. Note the name reuse: `mera_range` here is a plain function, while earlier it was a class. A name is just a label; what matters is the protocol the object it produces honours.

Generators

Use `yield` to produce values lazily, preserving state between calls - memory efficient for streams.

PYTHON

```
def mera_range(start,end):
    for i in range(start,end):
        yield i
```

PYTHON

```
for i in mera_range(15,26):
    print(i)
```

4. Generator expressions

4.1 Brackets versus parentheses

A generator expression is to a generator what a list comprehension is to a list — except it builds nothing. Compare the two neighbouring cells:

Only the brackets change: `[]` builds a list immediately, `()` builds a generator that computes each square the moment someone asks. The loop that follows prints `1, 4, 9, ...` up to `10000` in either case; the difference is *when* the work happens and how much memory it costs — hundreds of kilobytes for the list, a couple of hundred bytes for the generator object.

Properties worth memorising:

- **One-shot.** A generator expression can be consumed only once. `sum(gen)` takes everything; a second pass yields nothing.
- **Bare when sole argument:** `sum(i**2 for i in range(101))` is legal and idiomatic — the parentheses are implied.
- **Composable.** Generators feed generators without temporary lists, which is the engine behind the chaining example at the end of this notebook.

- **Same rules, fewer words.** Multiple `for` and `if` clauses work exactly as in comprehensions; only the eagerness changes.

Rule of thumb: reach for a list when you need indexing, `len()`, sorting or repeated re-use; reach for a generator expression when you will loop once and discard.

PYTHON

```
# List comprehension
L = [i**2 for i in range(1,101)]
```

PYTHON

```
gen = (i**2 for i in range(1,101))

for i in gen:
    print(i)
```

5. A practical generator: reading images lazily

5.1 One image in memory at a time

Machine-learning folders are a perfect generator use case: a training folder can hold thousands of images, and decoding them all into RAM *before* training starts is a fast route to an out-of-memory crash. The function in the next cell flips that script:

Read it as a patient conveyor belt. `os.listdir` merely *names* the files (cheap); `cv2.imread` decodes **one** file into a NumPy array; `yield` hands that array to the consumer and freezes the loop. The next pull decodes the next file. At any instant only a single decoded image exists inside the generator — everything else is still on disk as compressed file bytes.

Because a generator is an iterator, the consumer controls the pace. The demonstration cell pulls a few frames manually with `next(gen)` — each call reads exactly one more image. The same generator would work inside a `for` loop, or be wrapped in batching logic that yields chunks for a data loader.

Two practical notes. First, this cell depends on OpenCV (`cv2`) being installed. Second, it points at a fixed folder path from the original author's machine — on your computer, replace it with a path that actually contains images; any folder of `.jpg` files will do. The pattern, not the hard-coded path, is the lesson.

PYTHON

```
import os
import cv2

def image_data_reader(folder_path):

    for file in os.listdir(folder_path):
        f_array = cv2.imread(os.path.join(folder_path,file))
        yield f_array
```

PYTHON

```
gen = image_data_reader('C:/Users/91842/emotion-detector/train/Sad')

next(gen)
next(gen)

next(gen)
next(gen)
```

6. Benefits of using a generator

Four claims are usually made for generators, and this notebook demonstrates each one with actual code rather than slogans. Think of a generator as a delivery service that carries one package at a time along a route: cheap to set up, light on the vehicle, able to serve routes that never end, and easy to hand over to the next driver. In the language of Python, those four qualities have concrete names and concrete proofs.

First, a generator is far **easier to implement** than the equivalent iterator class pair — the next-but-one cell shows the two designs side by side. Second, it is dramatically **memory efficient**, holding only a paused frame instead of a full collection — the size comparison that follows is the single most persuasive number in this notebook. Third, it can represent **infinite streams** that lists could never store — an endless source of even numbers costs nothing. Fourth, generators **chain** into pipelines where values flow through several transformations without ever landing in an intermediate list — fib, squared, summed. The next four sections take each benefit one at a time, with a small experiment to back the claim.

6.1 Ease of implementation

Side by side, the comparison is brutal. The class version needs `mera_range` — a data holder with `__iter__` — plus `mera_iterator`, a cursor with `__iter__` and `__next__`: two classes, five methods, an explicit `start >= end` check and a manual `raise StopIteration`. The generator version is the four-line function you have already seen: a `for` loop and a single `yield`.

Everything in between is scaffolding you no longer maintain — no cursor attribute to keep in sync, no risk of forgetting `return self` in `__iter__`, no end-of-sequence logic to duplicate per iterator type. Python generates a tuned iterator object for you, tracks its position internally and raises `StopIteration` at exactly the right moment.

Ease here is not cosmetic. Fewer lines means fewer places for off-by-one bugs (`< end` versus `<= end` is a classic), straightforward testing — the body is an ordinary function — and readable intent: the word `yield` states *'produce these values over time'* on sight. When a four-line function fully replaces a two-class protocol, that is the language telling you which shape it prefers.

PYTHON

```

class mera_range:
    def __init__(self, start, end):
        self.start = start
        self.end = end

    def __iter__(self):
        return mera_iterator(self)

```

PYTHON

```

# iterator
class mera_iterator:
    def __init__(self, iterable_obj):
        self.iterable = iterable_obj

    def __iter__(self):
        return self

    def __next__(self):
        if self.iterable.start >= self.iterable.end:
            raise StopIteration

        current = self.iterable.start
        self.iterable.start += 1
        return current

```

6.2 Memory efficient

The measurement cell prints the punchline of this entire notebook:

Same hundred thousand integers — roughly **800 KB** as a list versus **200 bytes** as a generator. That is not a rounding difference; it is about four thousand times smaller, and the gap only grows with the data. A hundred million rows would be gigabytes as a list yet still a few hundred bytes as a generator object.

Where does the tiny footprint come from? A list stores every element — object headers, reference pointers, the works. A generator stores only its *paused frame*: the current value of the loop variable, the iterator it is reading from, and the line number to resume at. Elements are created, handed to you, and dropped; the generator never collects them.

Two caveats keep the claim honest. First, the 200 bytes measure the generator *object*, not the individual values flowing through it — memory stays flat, but each yielded value still exists while you hold it. Second, laziness only helps if you genuinely process-and-discard; collecting everything afterwards with `results = list(gen)` simply moves the big allocation into your own variable.

PYTHON

```
L = [x for x in range(100000)]
gen = (x for x in range(100000))

import sys

print('Size of L in memory', sys.getsizeof(L))
print('Size of gen in memory', sys.getsizeof(gen))
```

PYTHON

```
Size of L in memory 800984
Size of gen in memory 200
```

6.3 Representing infinite streams

Some sequences never end: sensor readings, counters, timestamps, the tail of a live log. A list cannot represent any of these — *all even numbers* has no final element, so the container can never finish building. A generator can, because it only ever promises the *next* value:

`while True` plus `yield` is the signature of an infinite generator. The demo cell creates `even_num_gen = all_even()` and pulls twice with `next()`, receiving `0` and `2`; the generator then sits paused at `n = 4`, ready indefinitely, and its memory use stays constant no matter how many values are consumed.

The flip side is discipline. Never drop an infinite generator into an ordinary `for` loop — `StopIteration` will never arrive, so the loop would run until you interrupt the kernel. Consume infinite streams deliberately: a fixed number of `next()` calls, a slice from `itertools.islice(gen, 100)`, or a `zip` with a finite partner that ends first. In other words, infinity is safe only when something else decides when to stop.

PYTHON

```
def all_even():
    n = 0
    while True:
        yield n
        n += 2
```

PYTHON

```
even_num_gen = all_even()
next(even_num_gen)
next(even_num_gen)
```

6.4 Chaining generators

The final cell is the payoff — three functions acting as three stations on one assembly line:

- `fibonacci_numbers(10)` yields ten Fibonacci numbers with a clean tuple-swap: `1, 1, 2, 3, 5, 8, 13, 21, 34, 55`.
- `square(...)` receives those values one at a time and yields each one squared.

- `sum(...)` pulls from `square`, which pulls from `fibonacci_numbers`, and totals everything: `4895`.

No intermediate list exists anywhere in that chain. Because every stage is a generator, each value is born at `fibonacci_numbers`, transformed in `square`, and consumed by `sum` — then all three forget about it. This *pull-based pipeline* is the backbone of a great deal of Python data work: `itertools`, lazy engines, and large file-processing scripts are all variations on this shape.

Two practical dividends. You can insert a new stage — a filter, a log, a batching step — by wrapping one more generator around the chain without rewriting anything. And memory stays flat no matter how many values flow through, because only one value occupies each stage at a time. When you catch yourself building throwaway lists in `for` loops, reach for a generator chain instead.

PYTHON

```
def fibonacci_numbers(nums):
    x, y = 0, 1
    for _ in range(nums):
        x, y = y, x+y
        yield x

def square(nums):
    for num in nums:
        yield num**2

print(sum(square(fibonacci_numbers(10))))
```

PYTHON

```
sum(square(fibonacci_numbers(10)))
```

Summary

A **generator** is an iterator you get by writing a normal function that uses `yield` instead of `return`. Calling that function produces a cold generator object; each pull runs the body to the next `yield`, freezes the local state, and hands over one value; when no `yield` remains, Python raises `StopIteration` for you. Generators beat hand-written iterator classes on every axis shown here: four lines instead of two classes, about 200 bytes instead of about 800 KB for the same hundred thousand values, the ability to model infinite streams with `while True`, and the freedom to chain stages — `sum(square(fibonacci_numbers(10)))` — so data flows without ever sitting in a list. Generator expressions, parentheses instead of brackets, give the same laziness in one line. Reach for a generator when you process values one at a time; use a list when you truly need them all at once.

An **iterable** owns data and knows how to produce iterators via `__iter__`; an **iterator** carries the cursor and provides `__iter__` (returning itself) and `__next__` (returning one value or raising `StopIteration`). Every iterator is iterable, but not the other way around — you can tell them apart with `dir()` and the presence of `__iter__` / `__next__`. A `for` loop is nothing more than one `iter()` call followed by repeated `next()` calls until `StopIteration` stops it silently; we proved it by rebuilding the loop ourselves with `while True` and `try/except`. Calling `iter()` on an iterator returns the same object, which is why loops never accidentally

rewind. Finally, we designed our own `range()` as a pair of classes — `mera_range` producing iterators and `mera_range_iterator` doing the stepping — the exact blueprint that generators automate in the next notebook.

Namespaces are the dictionaries that bind names to objects, and Python resolves every name through the **LEGB** chain — local, enclosing, global, built-in — stopping at the first hit and raising `NameError` when all four miss. Assignments always create or update the *local* name unless `global` (module level) or `nonlocal` (nearest enclosing function) says otherwise; built-ins sit at the bottom of the ladder and can be accidentally shadowed. Functions, being first-class objects, can be passed and returned — which makes a **decorator** possible: a wrapper-producing function that enhances another function without editing it. The `@name` line is shorthand for a reassignment that happens at definition time; robust wrappers forward `*args, **kwargs`, return the result, and preserve metadata with `functools.wraps`. Giving a decorator its own arguments simply adds a nesting level — factory, decorator, wrapper — exactly as `sanity_check(data_type)` demonstrates.

- `zip(a, b)` lazily pairs elements and stops at the shortest input; `strict=True` turns mismatches into errors.
- `zip(*rows)` unzips a list of tuples back into columns.
- `dict(zip(keys, values))` builds a lookup table in one line, and the same trick reverses a dictionary.
- `enumerate(seq, start=n)` yields `(index, item)` pairs and replaces `range(len(...))`.
- Both tools return single-use lazy iterators, so materialise them with `list()` when you need to reuse the result.
- Real data work — pairing names with totals and ranking with `enumerate` — becomes short and readable.
- `map(fn, items)` lazily transforms items and can take several iterables at once.
- `filter(pred, items)` lazily keeps passing items; `sum(1 for ...)` counts without a list.
- `reduce(fn, items[, initial])` eagerly folds a sequence into one value and needs an import.
- Prefer built-ins such as `sum`, `max`, and `math.prod` for standard folds, and comprehensions when you would write a one-off lambda.
- Real pipelines combine several machines: GST mapping, premium filtering, and inventory reduction over a products CSV.
- `map` and `filter` are single-use iterators; materialise them with `list()` before reusing.
- `b = a` is aliasing — one object with two names, sharing the same `id()`.
- Shallow copies (`.copy()`, `[:]`, `copy.copy()`) create a new outer container but share the inner objects.
- On flat lists a shallow copy is enough; on nested structures it leaks mutations to the original.
- `copy.deepcopy()` clones recursively so nothing is shared, at the cost of more time and memory.
- The reliable test is to compare the `id()` of the inner objects, not just the outer container.

Whenever a copy mysteriously changes the original, suspect a shallow copy over nested mutable data.

- A memory diagram makes it clear: shallow copies rebuild the shelf, deep copies rebuild the goods.
- Nested dictionaries and lists are shared by a shallow copy, so edits leak back to the original.
- `copy.deepcopy` gives full independence for nested structures you intend to modify.

- Tuples are reused by slicing; only a deep copy rebuilds the mutable objects they contain.
- The JSON round-trip is a readable deep copy for JSON types, but sets fail and tuples and integer keys change type.
- Decide with one question: can nested mutable data change? If yes, deep copy.

Mutability decides whether an object can be changed in place. Mutable objects — `list`, `dict`, `set`, `bytearray` — keep the same `id()` while their contents change, so every name pointing at them sees the edit. Immutable objects — numbers, strings, tuples, `frozenset`, `bytes` — are never modified; any "change" rebinds the name to a new object with a new `id()`.

The `id()` X-ray proved this for numbers, strings, lists and tuples, and showed that `+=` mutates a list but rebuilds a tuple. Hashability follows immutability, so only frozen values may be dictionary keys or set members. Finally, `==` compares values while `is` compares identity, and plain assignment creates an alias, not a copy.

Next up

`02_memory_behavior_experiments.ipynb` — the mutable-default-argument trap, caller-side side effects, and the cost of join versus concatenation.

- Arguments are passed by object reference; a parameter is another name for the caller's object.
- In-place mutation reaches the caller, while rebinding stays local.
- A mutable default argument is created once and shared across calls — use `None` plus an in-body initialisation.
- Defensive copying protects a caller when a function must mutate while processing.
- Immutability provides hashability, thread safety, and predictability.
- Build strings with `join`, not with `+=` in a loop.

These rules explain most why-did-my-data-change bugs and point directly to the safe patterns that prevent them.

- Positional arguments match by order and must precede keyword arguments in a call.
- Defaults make parameters optional; a required parameter cannot follow a default.
- `*args` packs extra positional arguments into an immutable tuple, even when empty.
- `print` is the classic `*args` function, and forwarding with `*args` passes arguments through unchanged.
- The signature order is positional, defaults, `*args`, keyword-only, `**kwargs`.
- Anything after a bare `*` is keyword-only and must be passed by name.
- `**kwargs` packs extra named arguments into a dictionary; `.get` reads optional fields safely.
- The full signature is positional, `*args`, keyword-only, `**kwargs`, in that fixed order.
- At the call site, `f(*list)` unpacks positionally and `f(**dict)` unpacks by name.
- A combined `packer(*args, **kwargs)` can count and forward both packs.
- `functools.partial` freezes some arguments to create a simpler callable.

- Applied to real data, these tools built an 18%-off discount lab and a dynamic receipt that ignores the flags it does not use.

Iterators and Iterables — Pythonic Thinking

1. Iteration, Iterables and Iterators

1.1 The two-object dance behind every loop

Iteration is the process of visiting items one after another — anything a `for` loop does is iteration. The code right below this cell is the plainest version: a list `num = [1, 2, 3]` and a `for` loop printing `1`, `2`, `3`.

An **iterable** is an object that *can* be iterated over: lists, tuples, strings, dicts, sets, `range` objects and files all qualify. Formally, an iterable knows how to produce an iterator when you call `iter()` on it — think of a cinema as the iterable: it can issue many tickets, show after show.

An **iterator** is the ticket holder: one object that walks a sequence step by step. It has exactly two powers — `iter()`, which simply returns itself, and `next()`, which returns the following value. When there is nothing left, `next()` raises `StopIteration`, the official end-of-the-road signal.

Two consequences matter in practice:

- **Iterators are lazy.** They never build the whole sequence in advance. The second cell confirms it: a list comprehension of almost ten thousand integers costs real memory, while `range(1, 10000000000)` costs almost nothing, because a `range` stores only a start, a stop and a step.
- **Iterators are one-shot.** Once exhausted, they are done — you must call `iter()` again to get a fresh walk.

Tiny taste: `it = iter([1, 2, 3])` creates a `list_iterator`; `next(it)` gives `1`, `next(it)` gives `2`, and one more call after `3` raises `StopIteration`. Those two functions are the entire protocol — the cells below simply prove it on real objects.

PYTHON

```
# Example
num = [1,2,3]

for i in num:
    print(i)
```

What is Iterator

An Iterator is an object that allows the programmer to traverse through a sequence of data without having to store the entire data in the memory

Semantics

Iterables produce iterators; iterators support one-pass, lazy iteration via **iter** and **next**.

PYTHON

```
# Example
L = [x for x in range(1,10000)]

#for i in L:
#    print(i*2)

import sys

print(sys.getsizeof(L)/64)

x = range(1,10000000000)

#for i in x:
#    print(i*2)

print(sys.getsizeof(x)/64)
```

What is Iterable

Iterable is an object, which one can iterate over

It generates an Iterator when passed to `iter()` method.

Semantics

Iterables can produce iterators; iterators implement **`iter()`** and **`next()`**, support lazy evaluation and one-pass iteration.

Why

Memory efficient for large streams; enables Python iteration protocol (for loops).

When to use

Large/infinite sequences, streaming data.

Common mistakes

Reusing exhausted iterators without recreating.

PYTHON

```
# Example

L = [1,2,3]
type(L)

# L is an iterable
type(iter(L))

# iter(L) --> iterator
```

Point to remember — the two rules

- **Every Iterator is also an Iterable.** An iterator implements `__iter__` and returns itself, so `iter(some_iterator)` is always allowed and gives back the very same object.
- **Not every Iterable is an Iterator.** A list, tuple, dict, set or string can be looped over, but it keeps no cursor of its own. Calling `next([1, 2, 3])` raises `TypeError`, because a list has no `__next__`.

	ITERABLE	ITERATOR
Must define	<code>__iter__()</code>	<code>__iter__()</code> and <code>__next__()</code>
Job	Produce iterators (a factory)	Track position and yield values one by one
Examples	<code>[1, 2, 3]</code> , <code>"hello"</code> , <code>range(10)</code> , <code>{1:2}</code>	<code>iter([1, 2, 3])</code> , an open file, a generator

Analogy time. A playlist is iterable — it can spawn players. Each player object is an iterator: it remembers the current track and moves forward. You can start several players from the same playlist, and each keeps its own position. Calling `iter()` on a player just resumes that same player rather than starting a new one — exactly what the `id()` experiment further down demonstrates.

Trick — recognise them with `dir()`

You never have to guess whether an object is iterable or an iterator — ask Python for its methods.

- `dir(obj)` without `__iter__` → the object is neither iterable nor an iterator. An `int` is like this: `for i in a:` on `a = 2` would raise `TypeError: 'int' object is not iterable`.
- `dir(obj)` with `__iter__` but without `__next__` → iterable, not an iterator. Lists, dicts and strings land here.
- `dir(obj)` with both `__iter__` and `__next__` → a full-fledged iterator. Generators and `list_iterator` objects look like this.

So the trick is a two-step checklist: look for `__iter__` first (the door), then look for `__next__` (the engine). The `dir(a)` and `dir(T)` calls in the next cells exist precisely for this comparison — the integer `a` shows no iteration methods at all, while the dict displays `__iter__` proudly (and `__getitem__`, which also helps loops). Everything else in those long dumps — `__add__`, `__eq__`, `__hash__` and friends — belongs to the object's other duties and can be ignored for now.

Mnemonic: **iterable** = has a door (`__iter__`); **iterator** = has a door and an engine (`__iter__` + `__next__`).

PYTHON

```
a = 2
a

#for i in a:
#    print(i)

dir(a)
```

PYTHON

```
T = {1:2,3:4}
dir(T)
```

2. How a `for` loop really works

2.1 Desugaring `for` into `iter()` + `next()`

Python's `for` statement is pure convenience. Behind the scenes every `for i in num:` is rewritten into a fixed four-step dance:

1. Ask the iterable for an iterator: `it = iter(num)`.
2. Pull a value: `value = next(it)`.
3. Bind the loop variable and run the loop body.
4. Repeat from step 2 — and the moment `next(it)` raises `StopIteration`, Python **silently** ends the loop and continues with whatever comes after it.

That is the whole feature. There is no hidden list, no index counter, no `range(len(...))` anywhere. This is why `for` works identically on lists, strings, dicts, sets, `range` objects, generators and your own custom classes: they all honour the same `__iter__` / `__next__` contract.

The first code cell after this header shows the friendly syntax. The second shows the raw version: `iter_num = iter(num)` fetches the iterator, then each `next()` call pulls one element — `1`, then `2`, then `3`. One extra `next()` call is included on purpose: the list is already exhausted, so it raises `StopIteration`, the very exception that normally ends a loop invisibly.

Two practical takeaways. First, mutating a collection while a `for` loop walks it is dangerous, because you are disturbing the walker's position in the middle of its journey. Second, any object that implements `__iter__` + `__next__` becomes instantly loopable — no inheritance from a base class required, just the protocol.

PYTHON

```
num = [1,2,3]

for i in num:
    print(i)
```

PYTHON

```
num = [1,2,3]

# fetch the iterator
iter_num = iter(num)

# step2 --> next
next(iter_num)
next(iter_num)
next(iter_num)
next(iter_num)
```

3. Writing our own `for` loop

3.1 `while True` plus `try/except StopIteration`

If a `for` loop is just `iter()` plus repeated `next()` until `StopIteration`, we can rebuild it in about eight lines. That is exactly what `mera_khudka_for_loop(iterable)` does:

Read the shape carefully. `iter(iterable)` is called **first**, on whatever the caller passed — a list, a `range`, a tuple, a set or a dict — so the function accepts any iterable, not just lists. The `while True` loop never tests a length or an index; the only exit door is the `StopIteration` exception. In Python, exceptions are not just error reports — they are normal control flow, and `StopIteration` is the official "no more items" message.

The demo cell then runs the custom loop over five containers, and for the dict it prints the keys `0` and `1`, because dicts iterate over keys by default — the same rule real `for` loops follow.

Notice what our version *cannot* do yet: it prints values instead of handing them back, so it cannot feed another loop as a data source. Fixing that is the job of generators, covered in the next notebook. For now, the point is proven: a `for` loop has no magic — just protocol plus exception handling.

PYTHON

```
def mera_khudka_for_loop(iterable):
    iterator = iter(iterable)

    while True:
        try:
            print(next(iterator))
        except StopIteration:
            break
```

PYTHON

```
a = [1,2,3]
b = range(1,11)
c = (1,2,3)
d = {1,2,3}
e = {0:1,1:1}

mera_khudka_for_loop(e)
```

4. `iter()` on an iterator returns itself

4.1 One object, two names

Here is a detail that confuses almost everyone the first time: calling `iter()` on something that is *already* an iterator does not create a second iterator — it returns the **same object**.

The experiment makes this visible. `iter_obj = iter([1, 2, 3])` creates a fresh `list_iterator`, and its `id()` is printed. Then `iter_obj2 = iter(iter_obj)` calls `iter()` on that iterator — and the printed `id()` is identical. Two names, one object.

Why must it work this way? Because `for` loops (and our custom loop from the previous section) *always* call `iter()` on their target before calling `next()`. If `iter()` on an iterator produced a brand-new iterator, every loop would silently restart from the beginning. The rule keeps things sane:

- `iter(iterable)` → a **new** iterator each time (a fresh walk over the same data),
- `iter(iterator)` → the **same** iterator; the walk continues where it stopped.

This also explains why iterators are legal iterables: they pass the first check (`__iter__` exists) while preserving their single cursor. So the relationship is subtle but clean — every iterator is an iterable, but an iterator's iterable-ness is a pass-through, never a rewind.

PYTHON

```
num = [1,2,3]
iter_obj = iter(num)

print(id(iter_obj), 'Address of iterator 1')

iter_obj2 = iter(iter_obj)
print(id(iter_obj2), 'Address of iterator 2')
```

5. Building our own `range()`

5.1 One iterable, one iterator

`range()` is the perfect teaching tool because its behaviour is familiar: `range(1, 11)` should produce `1, 2, ..., 10` every time you loop over it. Recreating it forces us to split the two roles we have been discussing into two separate classes.

`mera_range` (the iterable). Its only job is to hold the data — `start` and `end` — and to hand out iterators. Its `__iter__` method returns a brand-new `mera_range_iterator(self)`, so the same `mera_range` object can spawn fresh walkers, like a playlist spawning new players.

`mera_range_iterator` (the iterator). It keeps a reference back to the iterable and does the actual walking:

- `__iter__` returns `self` — the pass-through rule from the previous section,
- `__next__` checks `if self.iterable.start >= self.iterable.end:` and raises `StopIteration` when the range is finished,
- otherwise it returns the current number and advances the cursor.

The verification cells then show `type(x)` is `__main__.mera_range`, and `iter(x)` hands back a `mera_range_iterator` — iterable and iterator, cleanly separated.

One honest caveat about *this* particular implementation: the iterator advances the cursor by mutating `self.iterable.start` on the shared `mera_range` object, so numbers are consumed in place. After one complete pass, `x.start` already equals `x.end`, and a second loop over the same `x` would produce nothing. Built-in `range` avoids this by keeping its start and stop immutable and giving each iterator its own private copy of the position — a good exercise to attempt after this notebook.

PYTHON

```
class mera_range:
    def __init__(self, start, end):
        self.start = start
        self.end = end
    def __iter__(self):
        return mera_range_iterator(self)
```

PYTHON

```
class mera_range_iterator:
    def __init__(self, iterable_obj):
        self.iterable = iterable_obj
    def __iter__(self):
        return self
    def __next__(self):
        if self.iterable.start >= self.iterable.end:
            raise StopIteration
        current = self.iterable.start
        self.iterable.start += 1
        return current
```

Namespaces and Decorators — Pythonic Thinking

1. Scope and the LEGB rule

1.1 Python searches inside-out

A **scope** is the textual region of a program where a namespace is directly accessible. Python keeps several namespaces alive at once and resolves every bare name by walking outward through them in a fixed order — the famous **LEGB** rule:

ORDER	SCOPE	WHERE IT LIVES	TYPICAL NAMES
1	Local	The executing function's own frame	parameters, locally assigned names
2	Enclosing	Frames of any <i>outer</i> functions	variables of an enclosing <code>def</code>
3	Global	The current module's top level	module-level variables and functions
4	Built-in	The <code>builtins</code> module, always present	<code>print</code> , <code>len</code> , <code>max</code> , <code>True</code> , <code>Exception</code>

The search stops at the first hit. If all four miss, Python raises `NameError: name 'x' is not defined` — "not defined" really meaning "*not found in any accessible scope*".

Two corollaries drive most scope bugs. First, **reading** a name you have not defined locally is fine when an outer scope has it — the lookup simply falls through. Second, **assigning** a name inside a function always targets the *local* scope unless you say otherwise: `a = 3` inside `temp()` creates a brand-new local `a` and leaves the global one untouched. Even `a += 1` counts as an assignment, so Python marks `a` as local for the whole function — which is why that pattern fails with `UnboundLocalError` unless a local value already exists.

Shadowing is silent. The cells below walk each combination one at a time: local and global used together, the same name in both scopes, read-only access to a global, attempts to edit a global from inside a function, and finally the `global` keyword that makes the edit intentional.

PYTHON

```
# Local and global
# global var
a = 2

def temp():
    # local var
    b = 3
    print(b)

temp()
print(a)
```

PYTHON

```
# Local and global -> same name
a = 2

def temp():
    # Local var
    a = 3
    print(b)

temp()
print(a)
```

1.2 The built-in namespace

At the bottom of the LEGB ladder sits a namespace you did not create and cannot erase: **built-ins**. Every name Python provides out of the box — `print`, `len`, `max`, `min`, `range`, `sum`, `bool`, `dict`, `Exception`, all the types and error classes — is stored in the `builtins` module and is visible from anywhere without an import. The next cell proves it: `import builtins` followed by `dir(builtins)` dumps the entire catalogue into the output, and the tiny placeholder cells that follow mark the spot for your own exploration of that list.

Built-ins behave like any other name, with one crucial tweak in the search order: they are checked **last**. That gives you a legal but hazardous power — you can redefine them. The following demonstration rebinds the name `max` to a zero-argument function that prints `'hello'`. From that line onward, the name `max` in the global namespace shadows the built-in `max` for everything that resolves the name afterwards, so a later call like `max([1, 2, 3])` fails: the name now points at *your* function, which accepts no arguments.

The lesson is about respect for the LEGB ladder. Defining a variable or function whose name collides with a built-in — `list = ...`, `def print():`, `def max():` — silently hides the original for every look-up after it, and the hidden version is not restored until the session restarts or the name is reassigned. The conventions of the community — `type_`, `max_value`, or simply honest longer names — exist precisely to avoid squatting on shared public infrastructure: built-ins are always readable when you leave them alone, and quietly broken when you do not.

PYTHON

```
# built-in scope
import builtins
print(dir(builtins))
```

PYTHON

```
# renaming built-ins
L = [1,2,3]
print(max(L))
def max():
    print('hello')

print(max(L))
```

1.3 Enclosing scope and `nonlocal`

LEGB's **E** only exists when functions nest. In the next cell, `inner` is defined inside `outer`; when `inner` executes, it may read names from its own frame, then `outer`'s frame, then the module, then built-ins. That middle layer — the enclosing frame — is what makes **closures** possible: a function that keeps access to variables from the environment where it was born, even after that environment's function has returned.

Watch the first demo closely. `inner` prints `a`, yet neither `inner` nor `outer` defines `a` in that cell. The lookup sails past the inner local scope (nothing there), past the enclosing local scope (`outer` has no `a` either), and lands in the **global** scope, where the name `a` is still sitting with the value `5` left over from an earlier cell — which is why the stored output prints `5` before `outer function` and `main program`. In a clean session with no leftover global `a`, the same code would raise `NameError`. That is a healthy reminder that LEGB is a *fallback chain*, not a guarantee.

Then comes the `nonlocal` demo, where `inner` wants to *modify* `outer`'s `a`. A plain assignment would make `a` local to `inner` (shadowing), and `global a` would jump all the way to module level — both wrong targets. `nonlocal a` is the precise tool: "the `a` I mean lives in the nearest enclosing scope." With it, `a += 1` inside `inner` updates `outer`'s `a`, so both prints show `2`.

Rule of thumb: `global` targets the *module*; `nonlocal` targets the nearest enclosing *function*; and doing neither means a name you assign belongs to your *own* frame. The decorators in the second half of this notebook lean on exactly this enclosing-scope machinery.

PYTHON

```
# Enclosing scope
def outer():
    def inner():
        print(a)
    inner()
    print('outer function')

outer()
print('main program')
```

PYTHON

```
# nonLocal keyword
def outer():
    a = 1
    def inner():
        nonlocal a
        a += 1
        print('inner',a)
    inner()
    print('outer',a)

outer()
print('main program')
```

2. Decorators

2.1 Bolting behaviour onto a function from the outside

A **decorator** is a function that takes a function as input, wraps it with extra behaviour, and returns a new function. Nothing in the original function's source changes — the enhancement arrives from the outside, like slipping a book into a gift wrap without rewriting a single page.

This is only possible because of a feature you will prove in the very next cell: **functions are first-class citizens** — ordinary objects that can be stored in variables, passed as arguments and returned from other functions.

Python also ships a family of **built-in decorators** you will meet in object-oriented code: `@staticmethod` and `@classmethod` change how a method receives its first argument, `@property` turns a method into attribute-style access, and `@abstractmethod` marks a method that subclasses must implement. Everything else — timing, logging, validation, caching, web routing — is built by programmers as **user-defined decorators** shaped to their own needs.

Meanwhile, the `@` sign above a definition in later cells is not a new language feature at all: it is shorthand for a reassignment that executes the moment the `def` statement finishes. Understanding that one-line reassignment is ninety percent of the decorator story. The remaining ten percent is knowing that a robust wrapper forwards `*args, **kwargs`, returns the result, and preserves the original function's name and docstring.

PYTHON

```
def my_decorator(func):      # accepts a function
    def wrapper():          # defines the enhanced version
        print(...)         # extra behaviour before
        func()              # the original call
        print(...)         # extra behaviour after
    return wrapper          # hands back the enhanced version
```

2.2 First-class functions: the prerequisite

Before decorating anything, confirm the raw material. In Python, a function is a normal object: it has a name, lives in a namespace, can be stored in a container, and — the ingredient decorators need — can be **passed as an argument** and **returned as a result**.

The cell right after this one demonstrates the first half in the smallest possible frame:

`square` arrives as `func`, a plain parameter; calling `func(num)` is an ordinary call through a variable. If functions were compile-time blueprints you could not hand around, decorators would be impossible. Because functions are values, functions that *accept* functions are just utility functions — and functions that *return* functions are factories.

Three capabilities to tick off; each appears later in this notebook:

1. **Assign** a function to a name: `a = my_decorator(hello)`.
2. **Pass** a function into another: `modify(square, 2)`, and every decorator application.

3. **Return** a function from another: `my_decorator` returns `wrapper`; `sanity_check(int)` returns `outer_wrapper`.

A closure adds a fourth: the returned function can keep remembering variables from where it was created — which is precisely how `wrapper` holds onto `func` after `my_decorator` has already returned.

PYTHON

```
# Python are 1st class function

def modify(func,num):
    return func(num)

def square(num):
    return num**2

modify(square,2)
```

PYTHON

```
def modify(func, num):
    return func(num)

modify(square, 2) # 4
```

2.3 The wrapper pattern, by hand

Here is decoration with the training wheels still on — no `@` yet, just functions being honest about what they are.

`my_decorator(func)` defines an inner function called `wrapper` that does three things: print a line of asterisks, call `func()`, print another line of asterisks. Then it **returns** `wrapper` itself — not the result of calling it. Meanwhile `wrapper` keeps a live reference to the parameter `func`; even though `my_decorator(hello)` has finished executing, `wrapper` can still call `hello`. A function that remembers names from its enclosing scope after that scope has returned is called a **closure** — and it is the engine under every decorator.

Usage in the demo cells is explicit:

Notice that `hello` and `display` themselves were never edited — their bodies still print exactly what they always printed. Decoration is *outside-in*: behaviour is added by wrapping, never by modifying. Two details to file away: the wrapper defines no `return`, so decorated calls currently produce `None` (worth remembering when we reach the `timer` decorator), and the same `my_decorator` function formalises around two different functions, which is exactly the reusability that makes decorators a language feature rather than a one-off trick.

PYTHON

```
# simple example

def my_decorator(func):
    def wrapper():
        print('*****')
        func()
        print('*****')
    return wrapper

def hello():
    print('hello')

def display():
    print('hello nitish')

a = my_decorator(hello)
a()

b = my_decorator(display)
b()
```

PYTHON

```
a = my_decorator(hello)  # a IS wrapper, with func bound to hello
a()                     # asterisks, 'hello', asterisks

b = my_decorator(display) # same wrapper shape, different func
b()                       # asterisks, 'hello nitish', asterisks
```

2.4 The `@` syntax is a one-line reassignment

Manual decoration works, but writing `hello = my_decorator(hello)` under every function gets old fast. Python's answer is the `@` line:

This is **exactly equivalent** to defining `hello` normally and immediately running `hello = my_decorator(hello)`. Nothing else changes: the name `hello` ends up bound to the wrapper, so the later bare call `hello()` executes the decorated version, printing the asterisk fences just as in the hand-written demo.

Three details worth memorising:

- **Timing.** The decorator runs once, at *definition* time — right after the `def` line executes while the cell runs. Every later call to the function just invokes the already-wrapped result.
- **Position.** `@decorator` must sit directly above a `def` (or `class`). Stack them for composition; the bottom decorator wraps first.
- **Naming honesty.** After decoration, `hello.__name__` reports `'wrapper'`, not `'hello'`, unless the wrapper takes protective measures. The name you see in tracebacks, `help()` output and logs changes — a small identity crisis with real debugging costs, flagged again in two sections' time.

PYTHON

```
# Better syntax?
# simple example

def my_decorator(func):
    def wrapper():
        print('*****')
        func()
        print('*****')
    return wrapper

@my_decorator
def hello():
    print('hello')

hello()
```

PYTHON

```
@my_decorator
def hello():
    print('hello')
```

2.5 A decorator that earns its keep: @timer

Time to decorate something useful. The `timer` decorator wraps a function so that every call reports how long it ran:

Read the three moves. `wrapper(*args)` accepts **any number of positional arguments** — that single `*args` is what lets one decorator serve a zero-argument `hello()`, a one-argument `square(2)` and a two-argument `power(2, 3)` without knowing their signatures in advance. (This version does not forward `**kwargs`, so keyword-only calls would fail — a gap the robust template closes as well.) It then records `time.time()`, calls the original function, and prints the elapsed seconds alongside `func.__name__`, which still refers to the *original* function because the wrapper closed over it.

Applied via `@timer`, the three demo calls show the value immediately: `hello` reports roughly its `time.sleep(2)`, `square` reports about a second of sleeping, and `power` reports a near-zero runtime for `2**3 = 8`. One decorator, three unrelated functions, zero edits to their bodies — decoration as a drop-in measurement harness, reusable anywhere you want cross-cutting behaviour such as timing, logging, retries or access checks without repeating yourself.

PYTHON

```
# anything meaningful?
import time

def timer(func):
    def wrapper(*args):
        start = time.time()
        func(*args)
        print('time taken by', func.__name__, time.time()-start, 'secs')
    return wrapper

@timer
def hello():
    print('hello wolrd')
    time.sleep(2)

@timer
def square(num):
    time.sleep(1)
    print(num**2)

@timer
def power(a,b):
    print(a**b)

hello()
square(2)
power(2,3)
```

PYTHON

```
def timer(func):
    def wrapper(*args):
        start = time.time()
        func(*args)
        print('time taken by', func.__name__, time.time() - start, 'secs')
    return wrapper
```

2.6 The two classic wrapper pitfalls

The next cell is flagged as a problem checkpoint in the notebook — let us name the problems it points at. Both are visible in the `timer` wrapper we just read.

1. Swallowed return values. The wrapper calls `func(*args)` but never returns the result. For `hello()` that is harmless — a print-only function was returning `None` anyway. For `square(2)` it is a genuine bug: the decorated function now evaluates to `None`, so `result = square(2) * 10` would blow up even though the undecorated version worked. The fix is one keyword: `return func(*args)`.

2. Lost identity. After `@timer` is applied, `hello.__name__` reports `'wrapper'`, because the name lookup finds the wrapper function, not the original. Tracebacks, logs, `help()` output and any framework that inspects function names now see the wrong label. The standard remedy is `functools.wraps` applied *inside* the

decorator; it copies `__name__`, `__doc__` and friends from the original function onto the wrapper.

Neither pitfall is exotic — both appear constantly in real code, which is why the community template for a decorator looks like this: accept `*args, **kwargs`; do the pre-work; `result = func(*args, **kwargs)`; do the post-work; `return result` — with `@functools.wraps(func)` glued on top of the wrapper. Keep that five-line skeleton in mind and you can read almost any decorator you meet in production.

3. Decorators with arguments

3.1 Three nested functions: factory, decorator, wrapper

So far our decorators were applied as `@timer`. But what about `@sanity_check(int)` — a decorator that *itself* takes arguments? The trouble: Python evaluates `sanity_check(int)` **first**, and whatever it returns is what gets applied to the function below it. So `sanity_check(int)` must return... a decorator. And that decorator must return a wrapper. Hence three levels:

Trace `@sanity_check(int)` above a function: `sanity_check(int)` runs and returns `outer_wrapper`; Python then executes `square = outer_wrapper(square)`; `outer_wrapper` returns `inner_wrapper`; `square` ends up bound to that. On every call, `inner_wrapper` checks the first argument's type against `data_type` and either forwards the call or raises a `TypeError` with a custom message — a guard rail fitted around the function without touching its body. As written, the check assumes a single argument; a robust version would inspect all of `args`.

The cells that follow show the two halves of this idea. One sketches the intended usage with a `@checkdt(int)` style call, and the next holds the full `sanity_check` implementation — same three-level shape, different label. Two base configurations are registered from one recipe: `square` restricted to `int`, and `greet` restricted to `str`. Rule of thumb for the level count: **one extra argument in the `@` line means one extra nested function** — arguments in, decorator out, wrapper last.

PYTHON

```
@checkdt(int)
def square(num):
    print(num**2)
```

PYTHON

```
def sanity_check(data_type):
    def outer_wrapper(func):
        def inner_wrapper(*args):
            if type(*args) == data_type:
                func(*args)
            else:
                raise TypeError('Ye datatype nai chalega')
        return inner_wrapper
    return outer_wrapper

@sanity_check(int)
def square(num):
    print(num**2)

@sanity_check(str)
def greet(name):
    print('hello', name)

square(2)
```

Zip & Enumerate — Session 03: Pairing and Numbering

1. zip() — Interlocking the Teeth

`zip()` takes two or more iterables and produces tuples that pair their elements by position: the first items together, the second items together, and so on. It is named after a physical zipper that interleaves two rows of teeth into one track.

Like all the tools in this section, `zip` is **lazy** and returns an iterator, not a list. Iterate it once and it is spent; a second pass gives an empty result. Wrap it in `list()` or `dict()` to materialise it, and remember that materialising consumes it.

`zip` accepts any number of iterables, so `zip(names, marks, cities)` yields three-element tuples. The most common production uses are building dictionaries and ranking lists, both shown later in this notebook.

PYTHON

```
# 🐘 ZIP basics – teeth interlock!
names = ["Sana", "Vikram", "Priya"]
marks = [89, 76, 95]
pairs = zip(names, marks)
print("zip type :", type(pairs).__name__, "(L-A-G lazy hai!)")
print("list()   :", list(pairs))
print("dubara   :", list(pairs), "← ek-heart, dobara khaali! (nb1 wala trap!)")

# 3 lists bhi zipper-te:
cities = ["Delhi", "Pune", "Jaipur"]
print("\n3-zip  :", list(zip(names, marks, cities)))
# Expected OUTPUT:
# zip type : zip (L-A-G lazy hai!)
# list()   : [('Sana', 89), ('Vikram', 76), ('Priya', 95)]
# dubara   : [] ← ek-heart, dobara khaali! (nb1 wala trap!)
#
# 3-zip    : [('Sana', 89, 'Delhi'), ('Vikram', 76, 'Pune'), ('Priya', 95, 'Jaipur')]
```

PYTHON

```
names = ['Sana', 'Vikram', 'Priya']
marks = [89, 76, 95]
list(zip(names, marks)) -> [('Sana', 89), ('Vikram', 76), ('Priya', 95)]
```

2. Uneven Inputs, strict=True, and the Unzip Trick

When the inputs differ in length, `zip` follows the **shortest-list rule**: it stops as soon as any input is exhausted, silently discarding the extras. That is usually convenient, but it can hide a data bug — a column with missing rows quietly truncates the join.

Python 3.10 added `strict=True` as a guard: if the lengths differ, `zip` raises `ValueError` instead of truncating. Use it when the columns are *supposed* to be equal, so mismatches fail loudly. When you genuinely want to keep the longest input, `itertools.zip_longest` fills missing slots with a default.

The reverse operation is the **unzip trick**. Because `zip(*rows)` unpacks a list of tuples into separate arguments, it transposes them:

This one-liner turns row-oriented data back into columns, and is the backbone of transposing tables and rebuilding dictionaries.

PYTHON

```
# 🚀 UNEVEN Lists + strict guard! (3.10+)
short = [1, 2]
long = ["a", "b", "c", "d"]
print("normal zip : ", list(zip(short, long)), "← shortest-ka aangam!")

try:
    list(zip(short, long, strict=True))
except ValueError as e:
    print("strict=True 🚩:", e)

# unzip-trick: zip ka reverse? star-se kholo! ★
zipped = [("x", 1), ("y", 2), ("z", 3)]
letters, digits = zip(*zipped)
print("\nunzip letters:", letters)
print("unzip digits :", digits)
# Expected OUTPUT:
# normal zip : [(1, 'a'), (2, 'b')] ← shortest-ka aangam!
# strict=True 🚩: zip() argument 2 is shorter than argument 1
#
# unzip letters: ('x', 'y', 'z')
# unzip digits : (1, 2, 3)
```

PYTHON

```
rows = [('x', 1), ('y', 2), ('z', 3)]
letters, digits = zip(*rows) # ('x', 'y', 'z'), (1, 2, 3)
```

3. dict(zip(...)) — The Mini Dictionary Factory

Because `zip` yields `(key, value)` pairs, feeding it to `dict()` builds a mapping in a single expression: `dict(zip(airports, cities))`. This is the fastest way to combine two parallel lists into a lookup table, and it reads almost like English.

Pair it with the unzip trick and you can invert a dictionary. `zip(d.values(), d.keys())` produces swapped pairs, and `dict()` turns them into a reversed mapping. This works because dictionary keys are unique; if two original values are equal, the reversal keeps only the last one.

EXPRESSION	RESULT
<code>dict(zip(keys, values))</code>	A dictionary from two parallel lists
<code>dict(zip(d.values(), d.keys()))</code>	The same dictionary with keys and values swapped

Both forms are lazy on the `zip` side and materialise only when `dict()` consumes the pairs.

PYTHON

```
# 🍷 dict(zip(k, v)) – 10-second mini-dict factory! 🍷
airports = ["DEL", "BOM", "PNQ"]
cities   = ["Delhi", "Mumbai", "Pune"]
air_map = dict(zip(airports, cities))
print("factory dict :", air_map)

# KEYS-VALUES ko hi swap karna ho? – unzip rezip ID se bolo!
reverse = dict(zip(air_map.values(), air_map.keys()))
print("double-zipped:", reverse)

# Expected OUTPUT:
# factory dict : {'DEL': 'Delhi', 'BOM': 'Mumbai', 'PNQ': 'Pune'}
# double-zipped: {'Delhi': 'DEL', 'Mumbai': 'BOM', 'Pune': 'PNQ'}
```

4. enumerate() — The Queue-Token Machine

`enumerate(seq, start=0)` yields `(index, item)` pairs as it walks a sequence. It removes the classic `for i in range(len(seq))` pattern, where you fetch the value with `seq[i]`. With `enumerate` you get the index and the value together, which is clearer and less error-prone.

The `start` argument controls the first index — `start=1` gives human-friendly numbering, and any integer works. It is normally written as a keyword for readability.

A common use is finding positions: `[i for i, n in enumerate(names) if n == 'A']` returns every index where the value matches, without maintaining a counter by hand. Because `enumerate` returns a lazy iterator, wrap it in `list()` when you want to inspect it directly in a REPL or print statement.

PYTHON

```
# 🍷 ENUMERATE – queue-token machine! 🍷
items = ["chai", "samosa", "lassi"]
print("default (0-start):")
for i, item in enumerate(items):
    print(f" token#{i} → {item}")

print("\nstart=1 (human-style):")
for i, item in enumerate(items, start=1):
    print(f" token#{i} → {item}")

# Expected OUTPUT:
# default (0-start):
# token#0 → chai
# token#1 → samosa
# token#2 → lassi
#
# start=1 (human-style):
# token#1 → chai
# token#2 → samosa
# token#3 → lassi
```

5. enumerate vs range(len)

Both loops below produce identical output, but they send very different signals:

The first reaches for an index only to look the value back up; the second reads the value directly while carrying its position. The `enumerate` form is shorter, avoids off-by-one mistakes, and works on any iterable — including ones that have no `len()` or indexing, such as generators and file objects.

As a rule of thumb: if you ever type `range(len(...))`, stop and ask whether `enumerate` fits. It usually does. The only time you need a raw index range is when the index itself is the point and there is no sequence to walk.

PYTHON

```
# 🐞 enumerate vs range(len) - WHY FORMER GOAT 🐐
menu = ["idli", "dosa", "vada"]
print("C-style      :", [f'{i}:{menu[i]}' for i in range(len(menu))], "← old-school, clumsy")
print("Pythonic    :", [f'{i}:{x}' for i, x in enumerate(menu)], "← clean, furs NUMBERS+ITEMS saath!")

# enumerate with SENTINEL use - positions dhundna:
names = ["A", "B", "A", "A"]
hits = [i for i, n in enumerate(names) if n == "A"]
print("A-positions :", hits, "← index WITHOUT tracking-by-hand!")

# Expected OUTPUT:
# C-style      : ['0:idli', '1:dosa', '2:vada'] ← old-school, clumsy
# Pythonic     : ['0:idli', '1:dosa', '2:vada'] ← clean, furs NUMBERS+ITEMS saath!
# A-positions  : [0, 2, 3] ← index WITHOUT tracking-by-hand!
```

PYTHON

```
range(len(menu)) style : [f'{i}:{menu[i]}' for i in range(len(menu))]
enumerate style       : [f'{i}:{x}' for i, x in enumerate(menu)]
```

6. Real Dataset — The Student Leaderboard

This section applies both tools to `students_marks.csv` (50 students). After loading the rows, it builds two parallel lists: student names and their totals across five subjects.

`zip(names, totals)` pairs the columns, and `sorted(zip(totals, names), reverse=True)` turns them into a ranking. Placing the total first is deliberate: `sorted` compares tuples from the first element, so the highest total ranks first, and names break ties alphabetically.

Finally, `enumerate(ranked[:5], start=1)` adds human-friendly rank numbers. The combination — zip, sort, enumerate — is the standard recipe for any leaderboard, and you will reuse it whenever order matters.

SQL

```
# 🏠 REAL DATASET 🏠 Students Leaderboard: zip + enumerate ka combo! 🎓
import csv

def load_students():
    for path in ["../00_Resources/datasets/students_marks.csv",
                 "00_Python/00_Resources/datasets/students_marks.csv"]:
        try:
            with open(path, newline="", encoding="utf-8") as f:
                return list(csv.DictReader(f))
        except FileNotFoundError:
            continue
    raise FileNotFoundError("students_marks.csv nahi mila!")

students = load_students()
subjects = ["math", "science", "english", "hindi", "computer"]

names = [s["name"] for s in students]
totals = [sum(int(s[c]) for c in subjects) for s in students]
print("zip first-3 :", list(zip(names, totals))[:3])

# Leaderboard 🏠 sort DESC, enumerate RANK!
ranked = sorted(zip(totals, names), reverse=True)
print("\n 🏆 CLASS TOP-5:")
for rank, (t, n) in enumerate(ranked[:5], start=1):
    print(f" #{rank} {n:<12} total={t}")

# Expected OUTPUT:
# zip first-3 : [('Sneha_1', 289), ('Pooja_2', 249), ('Arjun_3', 348)]
#
# 🏆 CLASS TOP-5:
# #1 Manish_29 total=418
# #2 Ananya_20 total=401
# #3 Arjun_11 total=397
# #4 Sneha_24 total=393
# #5 Priya_44 total=390
```

Map, Filter & Reduce — Session 03: The Assembly Line

1. The Assembly Line

`map`, `filter`, and `reduce` are three machines on one production line.

MACHINE	JOB	CODE
Spray-paint station	The same job on every item	<code>map(paint, items)</code>
Quality gate	Keeps only items that pass	<code>filter(ok_weight, items)</code>
Blender end	Everything in, one result out	<code>reduce(mix, items)</code>

`map(function, iterable)` returns a lazy iterator that applies `function` to each item. It accepts more than two iterables: `map(lambda p, q: p * q, prices, qty)` pairs them like `zip` and transforms on the fly.

`filter(function, iterable)` returns a lazy iterator of the items for which `function` returns a truthy value. `reduce(function, iterable[, initial])` comes from `functools` and repeatedly combines two values until one remains — a sum, a product, a maximum, or any custom fold.

Remember the laziness rule: `map` and `filter` produce single-use iterators, so wrap them in `list()` to store or inspect them. `reduce` is eager and returns the final value directly.

PYTHON

```
# 🚗 MAP - spray-paint machine 🎨
prices = [100, 250, 799]
def gst(p):
    return round(p * 1.18, 2)

gstcd = map(gst, prices)
print("map type   :", type(gstcd).__name__, "(lazy machine)")
print("map result :", list(gstcd))

# Lambda version - one-liner bina function name ke:
halves = list(map(lambda x: x / 2, prices))
print("map lambda :", halves)

# MAP with TWO iterables - zipper-transform! chain-wise superpower ⚡
qty     = [2, 1, 5]
bills   = list(map(lambda p, q: p * q, prices, qty))
print("map 2-iter :", bills, "← 100*2, 250*1, 799*5 (pei-pe-heist!)")

# Expected OUTPUT:
# map type      : map (lazy machine)
# map result    : [118.0, 295.0, 942.82]
# map lambda    : [50.0, 125.0, 399.5]
# map 2-iter    : [200, 250, 3995] ← 100*2, 250*1, 799*5 (pei-pe-heist!)
```

2. filter() — The Quality Gate

`filter(predicate, iterable)` keeps every item for which `predicate(item)` is truthy. It is the selection half of the assembly line: given scores, `filter(lambda s: s >= 50, scores)` returns only the passing ones.

The result is a lazy iterator, not a list. In a REPL it prints as a `filter` object; wrap it in `list()` to see the values. Because it is single-use, materialising it consumes it.

When you only need a count rather than the values, do not build a list at all. `sum(1 for s in scores if s >= 50)` counts matches without allocating an intermediate collection — a clean and common idiom:

Prefer `filter` when a ready-made predicate function exists; prefer a comprehension when you would otherwise write a one-off lambda.

PYTHON

```
# 🍌 FILTER – quality-gate chhalni ✂️
scores = [42, 78, 55, 90, 33, 88]
passed = list(filter(lambda s: s >= 50, scores))
print("filter pass :", passed)

names = ["apple", "kiwi", "avanade", "mango"]
a_start = list(filter(lambda w: w.startswith("a"), names))
print("filter a-      :", a_start)

# filter lazily + len via sum-1 trick (list nahi banana hota):
count_pass = sum(1 for s in scores if s >= 50)
print("genexpr count:", count_pass, "← filter-list na banao, count chahiye to ye CLASSIC! ★")
# Expected OUTPUT:
# filter pass : [78, 55, 90, 88]
# filter a-    : ['apple', 'avanade']
# genexpr count: 4 ← filter-list na banao, count chahiye to ye CLASSIC! ★
```

PYTHON

```
list(filter(pred, data))      # keep the matching values
sum(1 for x in data if pred) # just count the matches
```

3. reduce() — The Blender at the End

`reduce(function, iterable[, initial])` folds a sequence into a single value. It applies `function` to the first two items, then to that result and the third item, and so on until one value remains: `reduce(add, [1, 2, 3, 4])` computes `((1+2)+3)+4 = 10`.

`reduce` lives in `functools` and must be imported. The optional third argument is the **initial** value, placed before the first item; it also lets `reduce` handle an empty sequence, which otherwise raises `TypeError`.

TASK	REDUCE	PREFERRED BUILT-IN
Sum	<code>reduce(lambda a, b: a + b, xs)</code>	<code>sum(xs)</code>
Product	<code>reduce(lambda a, b: a * b, xs)</code>	<code>math.prod(xs)</code> (3.8+)
Maximum	<code>reduce(lambda a, b: a if a > b else b, xs)</code>	<code>max(xs)</code>

Use `reduce` only when no built-in expresses the fold — for example, a custom accumulation over dictionaries. For sums, products, and extremes, the built-ins are clearer and faster.

PYTHON

```
# 🍷 REDUCE – blender-end rolling pin 🌀 (functools se import!)
from functools import reduce

nums = [1, 2, 3, 4]
total = reduce(lambda a, b: a + b, nums)
print("reduce sum :", total, "← but! sum(nums) pythonic hai:", sum(nums))

prod = reduce(lambda a, b: a * b, nums)
print("reduce prod :", prod, "← product ke liye math.prod() bhi hai 3.8+")

biggest = reduce(lambda a, b: a if a > b else b, nums)
print("reduce max :", biggest, "← but max(nums):", max(nums))

# reduce with INITIAL – emergency ki saulekaudit-style:
popularity = reduce(lambda a, b: a + b, [], 0)
print("with initial :", popularity, "(empty pe bhi survives! ✅)")
# Expected OUTPUT:
# reduce sum : 10 ← but! sum(nums) pythonic hai: 10
# reduce prod : 24 ← product ke liye math.prod() bhi hai 3.8+
# reduce max : 4 ← but max(nums): 4
# with initial : 0 (empty pe bhi survives! ✅)
```

Map/Filter vs Comprehension — Pythonic Boxing!

SCENE	BETTER CHOICE	WHY?
You would write <code>lambda x: ...</code>	Comprehension <code>[f(x) for x in ...]</code>	readable, explicit
A function already EXISTS (int, str.upper)	<code>map(fn, ...)</code> clean! — <code>list(map(int, s))</code>	no lambda needed
Complex condition+transform mix	Comprehension	both filter+map in one line
Lazy requirement	genexpr / map-filter chain — both lazy	a fresh tube

Guido's heart is on comprehensions 🧡 — use map/filter when "the function is ready".

4. Real Dataset — The Products GST Pipeline

This section applies all three machines to `products_catalog.csv` (30 products). It first loads the rows into memory with `csv.DictReader`, then extracts parallel lists of names and prices.

Three operations follow:

1. **map** an 18% GST function across prices to get taxed values, then pair them with names via `zip` for display.
2. **filter** the catalogue for premium items priced above 10,000, using a comprehension that filters and pairs in one line.

3. **reduce** the entire catalogue into one inventory figure, multiplying each product's price by its stock and accumulating the total.

The example shows the natural division of labour: **map** transforms, comprehensions filter, and **reduce** aggregates. Choosing the most readable form for each step is the Pythonic goal.

SQL

```
# 📁 REAL DATASET 📁 Products GST Pipeline! 🏭 (30 products)
import csv

def load_products():
    for path in [ "../00_Resources/datasets/products_catalog.csv",
                  "00_Python/00_Resources/datasets/products_catalog.csv" ]:
        try:
            with open(path, newline="", encoding="utf-8") as f:
                return list(csv.DictReader(f))
        except FileNotFoundError:
            continue
    raise FileNotFoundError("products_catalog.csv nahi mila!")

products = load_products()
names = [p["product_name"] for p in products]
prices = [float(p["price"]) for p in products]

# 1) GST map (18%) 📁 first-3 print (spray-paint station!)
gstcd = list(map(lambda p: round(p * 1.18, 2), prices))
print("GST first-3      :", list(zip(names, gstcd)[:3]))

# 2) FILTER: premium (>10,000) 📁 quality gate!
premium = [(n, int(p)) for n, p in zip(names, prices) if p > 10_000]
print("Premium (>10k)  :", premium)

# 3) REDUCE: poora inventory value (price 📁 stock sab ka total!)
from functools import reduce
inv_value = reduce(lambda acc, p: acc + float(p["price"]) * int(p["stock"]), products, 0)
print(f"Inventory value : ₹{inv_value:,.0f}")
# Expected OUTPUT:
# GST first-3      : [('Laptop', 56640.0), ('Smartphone', 17700.0), ('Headphones', 2360.0)]
# Premium (>10k)  : [('Laptop', 48000), ('Smartphone', 15000), ('Tablet', 22000), ('DSLR_Camera', 35000)]
# Inventory value : ₹16,629,090
```

5. Combining the Machines — Student Averages

The final example combines **map**, **filter**, **reduce**, and **zip** on **students_marks.csv**. For each student it sums five subject scores and divides by five to get an average, using **map** over the loaded rows.

- **map** builds the list of averages and the list of names.
- **zip** pairs names with averages for display.
- **filter** keeps only the distinction students, those with an average of at least 80.

- **reduce** sums the averages, which is then divided by the count to give the class average; a generator expression counts how many students scored above it.

This mirrors how real reports are assembled: transform, pair, select, aggregate. Each tool stays small, and the pipeline reads top to bottom.

SQL

```
# 📁 Students combo 📁 map/filter/reduce/zip ALL TOGETHER! 🎓🎵
def load_students():
    for path in ["../00_Resources/datasets/students_marks.csv",
                "00_Python/00_Resources/datasets/students_marks.csv"]:
        try:
            with open(path, newline="", encoding="utf-8") as f:
                return list(csv.DictReader(f))
        except FileNotFoundError:
            continue
    raise FileNotFoundError("students_marks.csv nahi mila!")

students = load_students()
SUBS = ["math", "science", "english", "hindi", "computer"]

# map: har student ka AVERAGE
avgs = list(map(lambda s: sum(int(s[c]) for c in SUBS) / 5, students))
names = list(map(lambda s: s["name"], students))
print("first-3 avgs :", list(zip(names, avgs))[:3])

# filter: distinction club (avg ≥ 80) 🏆
dist = list(filter(lambda na: na[1] >= 80, zip(names, avgs)))
print("distinction :", dist)

# reduce: class total-avg (all avgs ka mean)
class_avg = reduce(lambda a, b: a + b, avgs) / len(avgs)
print("class avg :", round(class_avg, 2))
print("above-avg :", sum(1 for a in avgs if a > class_avg), "students above class-normal")
# Expected OUTPUT:
# first-3 avgs : [('Sneha_1', 57.8), ('Pooja_2', 49.8), ('Arjun_3', 69.6)]
# distinction : [('Ananya_20', 80.2), ('Manish_29', 83.6)]
# class avg : 65.69
# above-avg : 27 students above class-normal
```

Copy Concepts — Session 03: Duplicate, Yet Not a Duplicate

1. Aliasing vs Copying

Assignment never copies. `b = a` simply binds a second name to the same object, so `id(a) == id(b)` and any mutation through either name is visible through the other. This is **aliasing**, and it is exactly what you want for large objects — copying everything on every assignment would be wasteful.

TERM	HOW IT IS CREATED	OBJECT RELATIONSHIP
Aliasing	<code>b = a</code>	Same <code>id</code> — one object, two names
Shallow copy	<code>a.copy()</code> , <code>a[:]</code> , <code>copy.copy(a)</code>	Outer <code>id</code> differs, inner <code>id</code> s are shared
Deep copy	<code>copy.deepcopy(a)</code>	Outer and inner <code>id</code> s all differ

The practical consequence: mutation of a mutable object is seen by every alias, while reassignment (`b = something_else`) only rebinds one name. The experiments that follow prove each row of this table with `id()`.

PYTHON

```
# 🐼 ALIASING – do nam, ek banda! (school fonts assignment danger zone 🚨)
a = [1, 2, 3]
b = a # COPY NAHI – sirf naya naam-tag!
print("a:", a, "| id:", id(a))
print("b:", b, "| id:", id(b), "← SAME ID! do naam, ek object")
b.append(99)
print("b.append(99) ke baad a:", a, "← DONO badle?! Haan – same object tha! 🤖")
# Expected OUTPUT:
# a: [1, 2, 3] | id: <koi-number>
# b: [1, 2, 3] | id: <SAME-number> ← SAME ID! do naam, ek object
# b.append(99) ke baad a: [1, 2, 3, 99] ← DONO badle?! Haan – same object tha! 🤖
```

2. Shallow Copy

A shallow copy builds a new outer container but reuses the inner references. For a **flat** list of numbers or strings that is completely safe: appending to the copy does not touch the original, because the two lists own separate pointer arrays.

All three common tools produce the same kind of result: slicing `original[:]`, the method `original.copy()`, and `copy.copy(original)`. Their outer `id()` values differ from the original, proving a new object was created.

The limitation appears only with nesting. If the elements are themselves mutable — lists, dictionaries, sets — the shallow copy still holds the *original* inner objects, so changes inside them leak to both the copy and the original. The next section demonstrates that trap.

PYTHON

```
# 🚀 SHALLOW copy – 3 tools, ek hi story! 📦🔗📦
original = [1, 2, 3]
c1 = original[:]           # tool#1: slicing
c2 = original.copy()       # tool#2: .copy()
import copy
c3 = copy.copy(original)   # tool#3: copy.copy()
print("original id:", id(original))
print("c1 id      :", id(c1), "← different (naya outer box ✅)")
print("c2 id      :", id(c2), "← different")
print("c3 id      :", id(c3), "← different")

# Flat list pe update safety – shallow YAHAN kaafi!
c1.append(99)
print("\nc1.append(99) → original:", original, "← SAFE! (flat list me shallow hi kaafi ✅)")
# Expected OUTPUT:
# original id: <n1>
# c1 id      : <n2> ← different (naya outer box ✅)
# c2 id      : <n3> ← different
# c3 id      : <n4> ← different
#
# c1.append(99) → original: [1, 2, 3] ← SAFE! (flat list me shallow hi kaafi ✅)
```

3. The Nested Trap

For a list of lists, a shallow copy is a half-measure. The outer list is new, but each inner list is shared.

`id(matrix) != id(shallow)`, yet `id(matrix[0]) == id(shallow[0])`.

The consequence is surprising the first time: `shallow[0].append(99)` also changes `matrix[0]`, because both names point at the same inner list. The outer containers are separate, but their contents overlap.

OPERATION	OUTER NEW?	INNER NEW?
<code>matrix.copy()</code>	Yes	No
<code>copy.deepcopy(matrix)</code>	Yes	Yes

The rule to remember: a shallow copy is safe when the contents are immutable or never mutated in place. As soon as nested mutable data can change, you need a deep copy.

Semantics

Assignment creates references (aliasing). Shallow copy copies top-level; nested objects shared. Deep copy recursively copies all objects.

Why matters

Prevents unintended side effects in mutable nested structures.

When to use

Shallow for flat; deep when independence needed.

Common mistakes

Assuming list slicing always gives deep copy for nested.

PYTHON

```
# 🍷 THE NESTED TRAP — shallow ka aaina-darr! 🐼⭐ (THE twist!)
matrix = [[1, 2, 3], [4, 5, 6]]
shallow = matrix.copy()           # outer NEW... par ANDAR?
print("outer ids differ:", id(matrix) != id(shallow), "✅")
print("INNER ids same? :", id(matrix[0]) == id(shallow[0]), "🐼 INNER SHARED!!")

shallow[0].append(99)             # inner furniture todte hain...
print("\nshallow  :", shallow)
print("matrix    :", matrix, "← MATRIX BHI BADAL GAYA?! stapled pages 💔")
# Expected OUTPUT:
# outer ids differ: True ✅
# INNER ids same? : True 🐼 INNER SHARED!!
#
# shallow   : [[1, 2, 3, 99], [4, 5, 6]]
# matrix    : [[1, 2, 3, 99], [4, 5, 6]] ← MATRIX BHI BADAL GAYA?! stapled pages 💔
```

4. Deep Copy

`copy.deepcopy(obj)` clones the object recursively: the outer container is new, and every nested mutable object is new as well. Mutating the deep copy therefore leaves the original completely untouched.

Because it walks the entire structure, `deepcopy` is slower and more memory-hungry than a shallow copy — a fair trade when correctness requires full independence. It also handles cycles and shared references correctly, preserving the shape of the original graph.

Use deepcopy when a structure contains nested mutables that may be modified, when you keep a pristine snapshot for comparison, or when passing configuration objects that called code might alter. For flat data, a shallow copy is faster and just as safe.

Semantics

Assignment aliases; shallow copy copies top level (nested shared); deep copy copies recursively.

5. The Real Test — Outer vs Inner

If the outer ids differ but any inner id matches, the copy is shallow and the matching parts are shared. If every id differs, the copy is deep. When in doubt, run the `id()` check on the actual structure rather than reasoning about it in your head. This is a common interview question, and the outer-versus-inner check is the answer that satisfies it.

Memory Diagrams Demo — Session 03: Mapping Copies in Memory

Read it as a rule: a shallow copy rebuilds the shelf labels but reuses the goods; a deep copy rebuilds both. Everything that follows is a variation on this picture.

Page 47 of 78

2. The Nested Dictionary Twist

Dictionaries make the sharing even easier to miss, because the nested mutable is hidden one level down. With `registry = {1001: {'name': ..., 'skills': [...]}}`, a `registry.copy()` gives a new outer dictionary, but `registry[1001]` — the inner dictionary — is shared, and so is its `skills` list.

Appending to `shallow_reg[1001]['skills']` therefore changes the original registry too. The outer ids differ, which is why the bug passes a casual check, but the inner ids match.

This is the classic trap in record processing: you copy a registry to avoid disturbing the source, edit a nested field, and discover the source moved with you. The fix is `copy.deepcopy`, which rebuilds the nested dictionary and list as well.

PYTHON

```
# 🚩 NESTED DICT twist – employee registry ka aaina-darr! 📁
import copy

emp_1001 = {"name": "Sai Das", "skills": ["python", "sql"], "dept": "Clothing"}
registry = {1001: emp_1001}

shallow_reg = registry.copy() # outer dict fresh...
print("outer different:", id(registry) != id(shallow_reg), "✅")
print("skills shared? :", id(registry[1001]["skills"]) == id(shallow_reg[1001]["skills"]), "👯")

# Shallow me inner list mutate 👯 – DONO registries me dikhta!
shallow_reg[1001]["skills"].append("excel")
print("\nshallow_reg :", shallow_reg[1001]["skills"])
print("registry      :", registry[1001]["skills"], "← OG me bhi aa gaya! 🤖")
# Expected OUTPUT:
# outer different: True ✅
# skills shared? : True 👯
#
# shallow_reg : ['python', 'sql', 'excel']
# registry    : ['python', 'sql', 'excel'] ← OG me bhi aa gaya! 🤖
```

3. Deep Copy for Registries

`copy.deepcopy(reg2)` produces a registry that shares nothing with the original: a new outer dictionary, a new inner dictionary, and a new `skills` list. Appending to the copy's skills leaves `reg2` exactly as it was, proven by comparing the inner list `id()` s.

The cost is proportional to the size of the structure, so deep copy is the right tool in two situations: when nested mutables may change, and when you need a guaranteed snapshot. For flat data, or for structures you only read, a shallow copy is cheaper and sufficient.

PYTHON

```
# 🚨 deepcopy – registry poorn-safe! ✂️📁
emp_2002 = {"name": "Kavya Gupta", "skills": ["java", "aws"]}
reg2 = {2002: emp_2002}
deep_reg = copy.deepcopy(reg2)
deep_reg[2002]["skills"].append("docker")
print("deep_reg:", deep_reg[2002]["skills"])
print("reg2      :", reg2[2002]["skills"], "← untouched! registry-divorce ❤️✅")
print("skills ids differ:", id(reg2[2002]["skills"]) != id(deep_reg[2002]["skills"]))
# Expected OUTPUT:
# deep_reg: ['java', 'aws', 'docker']
# reg2      : ['java', 'aws'] ← untouched! registry-divorce ❤️✅
# skills ids differ: True
```

5. The JSON Round-Trip Hack

A quick way to deep-copy JSON-compatible data is `json.loads(json.dumps(obj))`. Serialising to a string and parsing it back produces entirely new objects, so mutations to the clone do not affect the original. For configurations made of dictionaries, lists, strings, numbers, booleans, and null, this works well and reads clearly.

It is a hack, not a general solution. It only understands JSON types, it is slower than `copy.deepcopy`, and it silently changes some types. The next cell exposes those traps.

PYTHON

```
# 🚨 JSON-roundtrip jugaad – poora deep copy ek line me? 🤖
import json
config = {"app": "ShopKart", "features": ["search", "cart"], "limits": {"retries": 3}}

clone = json.loads(json.dumps(config))
clone["features"].append("checkout")
print("config:", config["features"])
print("clone  :", clone["features"])
print("independent:", id(config["features"]) != id(clone["features"]), "✅ jugaad works!")
# Expected OUTPUT:
# config: ['search', 'cart']
# clone  : ['search', 'cart', 'checkout']
# independent: True ✅ jugaad works!
```

6. JSON Traps — Types That Bite

Three specific traps make the JSON round-trip risky.

1. A **set** is not JSON serialisable, so `json.dumps` raises `TypeError`. The set must be converted to a list first.
2. A **tuple** becomes a list after the round-trip, so `(1, 2)` returns as `[1, 2]`.
3. Non-string dictionary **keys** are converted to strings, so `{42: 'answer'}` becomes `{'42': 'answer'}`.

ORIGINAL TYPE	AFTER JSON ROUND-TRIP
set	TypeError
tuple	list
int key 42	string key '42'

Use the JSON trick only when the data is plain JSON and you accept these conversions; otherwise prefer `copy.deepcopy`.

PYTHON

```
# ⚠️ JSON jugaad ke 3 TRAPS – types ka dhamaka! 🍷
weird = {"tuple": (1, 2), "intkey": 42, "set": {1, 2}}
try:
    json.loads(json.dumps(weird))
except TypeError as e:
    print("set trap 🚨:", e)

# Tuple silently LIST ban jata, int keys STR hote 🍷
partial = json.loads(json.dumps({"tuple": (1, 2), 42: "answer"}))
print("tuple→", type(partial["tuple"]).__name__)
print("keys →", list(partial.keys()), "| types:", [type(k).__name__ for k in partial.keys()])
# Expected OUTPUT:
# set trap 🚨: Object of type set is not JSON serializable
# tuple→ list
# keys → ['tuple', '42'] | types: ['str', 'str']
```

7. Decision Flowchart

Read the flowchart top to bottom. The decisive question is always whether **nested mutable data can change**; if it can, choose deep copy. This flowchart is a practical checklist you can apply at any copy site in real code.

PYTHON

```
Need a copy?
|- Flat (numbers/strings only) -> SHALLOW (fast: [:] or .copy())
|- Nested lists/dicts inside
|   |- Only reading the copy -> SHALLOW is fine (shared, read-only)
|   -| Inner mutation possible -> DEEPCOPY (copy.deepcopy)
|- Immutable shell (tuple) -> .copy() is a no-op; inner objects are still shared
-| JSON-safe types and a quick fix -> json round-trip (watch the type traps!)
```

Mutability vs Immutability — Whiteboard or Stone Tablet?

Real-World Hook: The 2 Brothers of Lunchbox-Service 🍱

- **Mutable Max** 🔥 = cottage cheese today, mixed vegetables tomorrow — the SAME lunchbox, only the curry changed (in-place, same id!).
- **Immutable Ivan** 🧊 = he will bring a NEW lunchbox for any new curry — every change = a new container (a new id!).

The Python family sorted:

🔥 MUTABLE (CHANGEABLE, SAME ID)	📦 IMMUTABLE (A NEW OBJECT EVERY CHANGE)
<code>list</code>	<code>int</code> , <code>float</code> , <code>bool</code>
<code>dict</code>	<code>str</code>
<code>set</code>	<code>tuple</code> , <code>frozenset</code>
<code>bytearray</code>	<code>bytes</code> , <code>NoneType</code>

Definitions

Mutable: the content can change IN-PLACE (object/address stays SAME) 🔥 **Immutable:** to change the content a NEW object is created (address changes!) 📦

1. The id() X-ray — Watching Immutability Live

1.1 Numbers: Every Arithmetic Step Builds a New Object

`id()` returns the memory address of an object as an integer. Because numbers are immutable, `x = x + 1` does **not** modify the object `100`; it creates a new object `101` and rebinds the name `x` to it. The old `100` may still exist elsewhere, completely untouched.

A useful mental model: the name is a label, the object is a box. For immutable types the label keeps jumping to a new box; for mutable types the label stays glued to the same box and only the inside changes.

The second half of the experiment reveals **small-integer interning**, a CPython optimisation: the integers `-5` through `256` are pre-created and reused, so two assignments to `5` often point at the *same* object. Larger values such as `1000` are freshly allocated each time, so two equal values live at different addresses.

EXPRESSION	RESULT	WHY
<code>x = 100; x = x + 1</code>	new <code>id()</code>	arithmetic builds a new int
<code>int("5") is int("5")</code>	usually <code>True</code>	small ints are cached
<code>int("1000") is int("1000")</code>	<code>False</code>	big ints are fresh objects

⚠️ Interning is an implementation detail. Compare values with `==`, never with `is`.

PYTHON

```
# 🍌 id() X-ray LESSON #1 – numbers hamesha naya object! 📦
x = 100
print("x pehle :", x, "| id:", id(x))
x = x + 1 # math kiya → NAYA object bana
print("x baad :", x, "| id:", id(x), "← ID BADAL GAYA! 📦 (imm)")

# 🌟 BONUS: chhote ints interned hote – id kabh-kabhi SAME dikhta (CPython jugaad!)
a, b = int("5"), int("5") # int("...") se FRESH parse – folding nahi!
print("\n5 vs 5 (small-int intern):", a is b, "← chhote ints cached details 😊")
a, b = int("1000"), int("1000")
print("1000 vs 1000:", a is b, "← bade ints fresh objects – dono same-value, different-identity!")
# Expected OUTPUT:
# x pehle : 100 | id: <n1>
# x baad : 101 | id: <n2> ← ID BADAL GAYA! 📦 (imm)
#
# 5 vs 5 (small-int intern): True ← chhote ints cached details 😊
# 1000 vs 1000: False ← bade ints fresh objects – dono same-value, different-identity!
```

1.2 Strings: Concatenation and Methods Return New Objects

A string is immutable, so `s = s + " pila dijiye"` builds a brand-new string object and rebinds `s`. The original object stays frozen, which is why building a string inside a loop is expensive: every `+` allocates and copies the whole result.

Exactly the same rule applies to string **methods**: `s.upper()`, `s.strip()` and `s.replace(...)` return **new** strings and never modify the receiver. A method that appears to "change" a string is really handing you a new object.

This is a safety feature: because a string can be shared freely between many parts of a program, no function can accidentally corrupt it. It is also exactly why strings are legal dictionary keys and set members — a rule you will meet again in section 3.

PYTHON

```
# 🍌 id() X-ray LESSON #2 – strings ka tablet-roe 📦
s = "chai"
print("s pehle :", s, "| id:", id(s))
s = s + " pila dijiye"
print("s baad :", s, "| id:", id(s), "← concat pe NAYA string bana 📦")

# String METHOD bhi nayata deta – OG never change
big = s.upper()
print("\nmethod result:", big)
print("OG unchanged :", s, "← immutability OP 😎")
# Expected OUTPUT:
# s pehle : chai | id: <n1>
# s baad : chai pila dijiye | id: <n2> ← concat pe NAYA string bana 📦
#
# method result: CHAI PILA DIJIYE
# OG unchanged : chai pila dijiye ← immutability OP 😎
```

```
PYTHON

s = "chai"           # object A at address id1
s = s + " pila dijiye" # object B at address id2, A still frozen
s.upper()            # object C, s still points to B
```

2. Mutable Containers and the += Double Standard

2.1 Lists Change In Place; Tuples Are Rebuilt

A list is mutable: `append`, `extend`, `pop`, `sort` and item assignment all change the object **in place**, so its `id()` is unchanged. The statement `lst += [100]` is the interesting case. Python calls the in-place operator `__iadd__` on the list, which extends the very same object, so the `id()` stays the same. That is efficient, but dangerous when another name points at the same list (aliasing): both names observe the change.

A tuple is immutable, so `t += (4,)` has no in-place operation to call. Python falls back to `t = t + (4,)`, which creates a **new** tuple and rebinds `t`. The address changes, and any other name still points at the old, unchanged tuple.

OPERATION	TYPE	IN PLACE?	ID()
<code>lst.append(x)</code>	<code>list</code>	yes	same
<code>lst += [x]</code>	<code>list</code>	yes	same
<code>t += (x,)</code>	<code>tuple</code>	no	new
<code>s += "x"</code>	<code>str</code>	no	new

 Rule of thumb: for mutable containers `+=` mutates; for immutable types it rebinds. Look at the object's `id()` whenever you are unsure.

PYTHON

```
# 🚀 id() X-ray LESSON #3 – LIST: whiteboard live! 🔥
lst = [1, 2, 3]
print("lst pehle :", lst, "| id:", id(lst))
lst.append(99)
print("append ke baad:", lst, "| id:", id(lst), "← id SAME! in-place modify 🔥")
lst += [100] # += LIST PE BHI IN-PLACE!
print("lst += [100] :", lst, "| id:", id(lst), "← STILL SAME!")

# TUPLE pe += ?? REBUILD hota hai (imm yaad!) 📦
t = (1, 2, 3)
id_before = id(t)
t = t + (4,)
print("\ntuple += ke baad:", t, "| id changed?", id(t) != id_before, "← REBUILD hua 😬")
# Expected OUTPUT:
# lst pehle : [1, 2, 3] | id: <n1>
# append ke baad: [1, 2, 3, 99] | id: <n1> ← id SAME! in-place modify 🔥
# lst += [100] : [1, 2, 3, 99, 100] | id: <n1> ← STILL SAME!
#
# tuple += ke baad: (1, 2, 3, 4) | id changed? True ← REBUILD hua 😬
```

3. Hashability — The Members-Only Rule

3.1 Only Frozen (Immutable) Objects May Be Keys

A Python `dict` key and a `set` member must be **hashable**. Hashing turns a value into a number that the table uses for fast lookups, and that number must never change while the object sits inside the container. A mutable object could change after insertion and silently corrupt the lookup table, so Python forbids it.

Consequently:

- **hashable:** `str`, `int`, `float`, `bool`, `tuple` (only if every element is itself hashable), `frozenset`, `None`.
- **not hashable:** `list`, `dict`, `set`, `bytearray`.

A subtle trap is a **tuple that contains a list**: the tuple looks immutable, but one of its elements is mutable, so the whole thing is unhashable and cannot be a key. Use a nested tuple or a `frozenset` instead. The practical summary is that hashability tracks immutability closely, and `hash(obj)` raises `TypeError: unhashable type` the moment the rule is broken.

PYTHON

```
# 🚩 HASH-KEY rule – dict ka members-only club! 🚩
phone = {"Ravi": "98765", "Meena": "91234"} # str keys (imm) ✓
print("str keys work:", phone)

bad_key = (["a", "b"], 2) # List andar – unhashable!
try:
    hash(bad_key)
except TypeError as e:
    print("tuple-with-list 🚩:", e, "← dict-key NAKAAM!")

# Fix: immutable versions use karo!
good = ("a", "b"), 2
print("pure-tuple hash OK:", hash(good) is not None, "→ dict-key valid ✓")
boot = {[1, 2]: "nope"} if False else "sets pe hash nahi – frozenset YES ✓"
print(boot)

# Expected OUTPUT:
# str keys work: {'Ravi': '98765', 'Meena': '91234'}
# tuple-with-list 🚩: unhashable type: 'list' ← dict-key NAKAAM!
# pure-tuple hash OK: True → dict-key valid ✓
# sets pe hash nahi – frozenset YES ✓
```

4. Identity vs Equality — `is` versus `==`

4.1 Two Names, One Object — Aliasing and the Meaning of `is`

Python gives you two comparison operators that answer two different questions:

- `==` asks **"are these two values equal?"** It calls `__eq__` and compares content, so `[1, 2, 3] == [1, 2, 3]` is `True`.
- `is` asks **"are these literally the same object?"** It compares `id()`, so `[1, 2, 3] is [1, 2, 3]` is `False` because two separate lists were built.

Assignment never copies: `z = x_list` makes `z` a second label for the *same* list, so `z is x_list` is `True`. Editing through either name changes both, which is the aliasing behaviour that bites real programs.

Use `is` in exactly two everyday situations: comparing against `None` (`if x is None`) and checking a sentinel object. For everything else — numbers, strings, containers — use `==`. Small-int and string interning can make `is` accidentally return `True`, so relying on it for value comparison produces code that breaks as soon as the data gets larger.

PYTHON

```
# 🍌 is vs == ka pehla jhala - same-value ≠ same-object! 🙅‍♂️
x_list = [1, 2, 3]
y_list = [1, 2, 3]
print("x == y:", x_list == y_list, "← VALUES same ✅")
print("x is y:", x_list is y_list, "← OBJECTS alag (do dabbas, same subzi!)")

# is = identity-train (id matching), == = equality-punch (values matching)
z = x_list
print("\nz = x_list; z is x:", z is x_list, "← aliasing - ab SAME dabba hai 🍌🍌")
# Expected OUTPUT:
# x == y: True ← VALUES same ✅
# x is y: False ← OBJECTS alag (do dabbas, same subzi!)
#
# z = x_list; z is x: True ← aliasing - ab SAME dabba hai 🍌🍌
```

Memory Behavior Experiments — Session 03: What Happens Inside Functions

1. Passing Objects into Functions

When you call a function, Python binds each parameter to the argument object without copying it. Inside the function the parameter and the caller's variable refer to the same object.

Two operations then behave very differently:

- **In-place mutation** (`.append`, `.extend`, `[i] = x`) changes the shared object, so the caller sees the change.
- **Rebinding** (`param = []`) only points the local name somewhere new. The caller's name still refers to the original object.

INSIDE THE FUNCTION	CALLER AFFECTED?
<code>lst.append(x)</code>	Yes — shared object changed
<code>lst = []</code>	No — only the local name moved

This is the office-whiteboard analogy: if you erase the board, everyone sees it; if you hang a different board on your own wall, the original is untouched.

PYTHON

```
# 🧪 EXPERIMENT #1 – Caller ka whiteboard effect! 📌

def add_task(task_list):
    task_list.append("new office task")    # in-place – whiteboard edit 🍷

def reset_board(task_list):
    task_list = []                        # REASSIGN – sirf LOCAL tag bana!

office_board = ["water-plants", "check-gate"]
add_task(office_board)
print("add_task baad :", office_board, "← caller CHANGED! (in-place 🍷)")
reset_board(office_board)
print("reset baad   :", office_board, "← caller UNTOUCHED! (reassignment = local 📦)")

# Expected OUTPUT:
# add_task baad : ['water-plants', 'check-gate', 'new office task'] ← caller CHANGED! (in-place 🍷)
# reset baad   : ['water-plants', 'check-gate', 'new office task'] ← caller UNTOUCHED! (reassignment = local 📦)
```

2. The Mutable Default-Argument Trap

A default argument is evaluated **once**, when the `def` statement executes — not on each call. For immutable defaults like numbers or strings that is harmless. For a mutable default such as `[]` or `{}`, the same object is reused across every call.

So `def enroll(student, subjects=[])` creates one list when the function is defined, and every call that omits `subjects` appends to that same list. The first call returns `['math']`, the second `['math', 'math']`, and so on — values from unrelated calls accumulate.

The fix is the **None-guard** pattern: use `None` as the default and create a fresh list inside the body when nothing was passed.

Rule: never use a mutable literal as a default. Use `None` and initialise inside.

PYTHON

```
# 🧪 EXPERIMENT #2 – THE Mutable Default-Arg Trap! (LEGEND ★)

def enroll(student, subjects=None):    # 🧪 default list EK baar banti (def-time!)
    subjects.append("math")
    return subjects

print("call 1:", enroll("Sana"))
print("call 2:", enroll("Ravi"), "← RAVI SIRF math to chahiye... pehli-wali value shared?! 🤖")
print("call 3:", enroll("Meena"))

# Expected OUTPUT:
# call 1: ['math']
# call 2: ['math', 'math'] ← RAVI SIRF math to chahiye... pehli-wali value shared?! 🤖
# call 3: ['math', 'math', 'math']
```

PYTHON

```
def enroll(student, subjects=None):
    if subjects is None:
        subjects = []          # fresh list every call
    ...
```

2.1 The None-Guard Fix

`enroll_fixed(student, subjects=None)` starts each call with `subjects` set to `None`, then replaces it with a brand-new empty list before appending. Because the new list is created inside the function body, it lives only for that call.

The result is the expected behaviour: each student gets `['math']`, with no sharing between calls. The guard costs almost nothing and makes the function's contract obvious to readers.

Apply this pattern to every mutable default — lists, dictionaries, and sets. Immutable defaults such as `0`, an empty string, or `None` are safe as they are.

PYTHON

```
# ✅ THE FIX – None-guard pattern (industry standard!)
def enroll_fixed(student, subjects=None):
    if subjects is None:
        subjects = []          # har call FRESH list ✨
        subjects.append("math")
    return subjects

print("call 1:", enroll_fixed("Sana"))
print("call 2:", enroll_fixed("Ravi"), "← ab fresh-plate har bande ko! ✅")
print("call 3:", enroll_fixed("Meena"))
# Expected OUTPUT:
# call 1: ['math']
# call 2: ['math'] ← ab fresh-plate har bande ko! ✅
# call 3: ['math']
```

3. Defensive Copying

Sometimes a function *should* modify its input, and sometimes it must protect the caller. When the caller's data must stay intact, the function can take a defensive copy at the top of its body: `scores = scores.copy()`.

After that line the function works on its own list, so appends and edits never reach the caller. This is the standard way to combine needing to mutate during processing with not surprising the caller.

Choose deliberately: mutate in place when the side effect is intended and documented, otherwise copy first. Silent in-place mutation of a caller's argument is one of the most common sources of hard-to-trace bugs.

PYTHON

```
# 🧪 EXPERIMENT #3 – Defensive copy (caller-protect pattern)
def process_scores(scores):
    scores = scores.copy()          # apni whiteboard banwalk, OG safe!
    scores.append(100)
    return scores

exam_scores = [80, 85, 90]
result = process_scores(exam_scores)
print("result      :", result)
print("exam_scores:", exam_scores, "← caller SAFE (defensive .copy() 🧪)")
# Expected OUTPUT:
# result      : [80, 85, 90, 100]
# exam_scores: [80, 85, 90] ← caller SAFE (defensive .copy() 🧪)
```

4. Immutability’s Superpowers

Immutable objects — strings, numbers, tuples, frozensets — cannot be changed after creation. That constraint buys three concrete benefits.

POWER	WHAT IT GIVES	EXAMPLE
Hashability	Safe dictionary keys and set members	<code>{('Sana',): 90}</code>
Thread safety	Can be shared freely, no one can corrupt it	Shared config strings
Predictability	A function cannot alter it in place	Caller peace of mind

The experiments show this directly: `s.upper()` returns a new string while the original keeps its `id()`, proving it was never modified. Immutability is not a limitation to work around; it is what makes dictionary keys reliable and sharing safe. Mutables trade that safety for fast, flexible edits.

PYTHON

```
# 🧑‍🔬 Immutability superpowers – STRINGS demo
s = "hero"
hero_id = id(s)

# string "badalna" = naya hi banana – but predictable, hashable, shareable-safe!
upper_s = s.upper()
print("upper      :", upper_s, "| original:", s)
print("OG id same? :", id(s) == hero_id, "← kabhi modify hi nahi hui (stone-tablet! 🧑‍🔬)")

# dict-key ho sakta – immutable superpower #2 LIVE:
scores = {("Sana",): 90}
print("tuple-keys OK:", scores)
# Expected OUTPUT:
# upper      : HERO | original: hero
# OG id same? : True ← kabhi modify hi nahi hui (stone-tablet! 🧑‍🔬)
# tuple-keys OK: {('Sana',): 90}
```

5. String Building: Concat Loop vs join

Strings are immutable, so `badge += p` inside a loop builds a **new** string on every iteration. For a short list the cost is invisible, but for thousands of parts the repeated allocation dominates.

The efficient idiom is `''.join(parts)`, which walks the parts once and allocates a single result string. Use it whenever you assemble text from a collection:

If you need separators or formatting, build a list of pieces and join once. As a habit, treat `join` as the default and `+=` in a loop as a smell.

PYTHON

```
# ⚡ EXPERIMENT #4 - concat-loop vs join (string-surgery cost!)
parts = ["chai", "samosa", "lassi"]

# WAY-A: concat-loop - har step NEW string (n strings = n objects!) 🙄🙄🙄
badge = ""
for p in parts:
    badge += p + ", "
print("concat loop :", badge)

# WAY-B: join - EK hi final string create 🤖 (efficient!)
badge2 = ", ".join(parts)
print("join      :", badge2, "← ek-shot memory-friendly way!")
# Note: chhote sizes pe farak chupcha; LOOPS me built-up strings ke liye join = golden rule ★
# Expected OUTPUT:
# concat loop : chai, samosa, Lassi,
# join      : chai, samosa, Lassi ← ek-shot memory-friendly way!
```

PYTHON

```
slow : total = ''
      for p in parts:
          total += p
fast : total = ''.join(parts)
```

6. Immutability's Three Superpowers

POWER	WHAT IT GIVES	EXAMPLE
Hashability	Can be used as dict keys or set members	<code>{('Sana',): 90}</code>
Thread safety	Safe to share across threads	Configuration strings
Predictability	No in-place changes inside functions	Stable caller data

Mutable types have their own advantages — fast in-place edits, easy growth — and are the right choice when data genuinely changes. The skill is choosing deliberately: immutable by default for shared or key-like data, mutable when you need to edit in place. This table is the why-bother answer to the whole topic.

Args & Kwargs — Session 03: Flexible Function Signatures

1. Positional vs Keyword Arguments

Python matches arguments to parameters in two ways. **Positional** arguments are matched by order: the first argument goes to the first parameter, and so on. **Keyword** arguments are matched by name: `city='Jaipur'` goes to the `city` parameter regardless of position.

Defaults make parameters optional. In `def introduce(name, city, hobby='coding')`, `hobby` may be omitted. The rules are strict:

- Positional arguments must come before keyword arguments in a call.
- A required parameter cannot follow one with a default in a definition.
- Missing required arguments raise `TypeError`.

These rules exist so that every call can be resolved unambiguously. The experiments below show the successful styles and the missing-argument error.

PYTHON

```
# 🍌 Positional vs Keyword calls – style-boxing!
def introduce(name, city, hobby="coding"):
    print(f"I am {name} from {city}. I love {hobby}.")

introduce("Sana", "Delhi")           # positional + default
introduce("Ravi", "Pune", "cricket") # 3 positional
introduce(name="Meena", hobby="dance", city="Jaipur") # pure-key (order-free!)

try:
    introduce("Kavi")                # city missing!
except TypeError as e:
    print("missing-arg 🚨:", e)

# Expected OUTPUT:
# I am Sana from Delhi. I Love coding.
# I am Ravi from Pune. I Love cricket.
# I am Meena from Jaipur. I Love dance.
# missing-arg 🚨: introduce() missing 1 required positional argument: 'city'
```

PYTHON

```
f(1, 2, x=3) # valid: positionals first, then keywords
f(x=3, 1)   # invalid: positional after keyword
```

2. *args — The Flexible Bag

Placing `*args` in a signature tells Python to collect any extra positional arguments into a tuple called `args`. The name `args` is only a convention; any name after the star works, but `args` is idiomatic.

`def order_summary(customer, *items)` fixes one required parameter and accepts zero or more items after it. Whether the caller passes one item or ten, the function receives a tuple and can loop over it, take its length, or sum it.

This is the flexible-bag idea: the caller packs as many items as needed, and the function unpacks the bag with ordinary tuple operations. It is how libraries build APIs that grow without breaking old calls.

PYTHON

```
# 🍷 `*args` – flexible thali! 🍷 (chemist-window Live!)
def order_summary(customer, *items):
    print(f"{customer} ordered {len(items)} items:")
    for i, item in enumerate(items, start=1):
        print(f"  # {i}: {item}")

order_summary("Sana", "paracetamol")
print()
order_summary("Ravi", "bread", "milk", "eggs", "butter")
# Expected OUTPUT:
# Sana ordered 1 items:
#   #1: paracetamol
#
# Ravi ordered 4 items:
#   #1: bread
#   #2: milk
#   #3: eggs
#   #4: butter
```

3. `*args` Is Always a Tuple

Whatever the caller passes, `args` is a tuple — never a list. That matters because tuples are immutable: you cannot append to `args`, and you should not try. When you need a list, convert explicitly.

CALL	ARGS VALUE	TYPE
<code>f(10, 20, 30)</code>	<code>(10, 20, 30)</code>	tuple
<code>f(7)</code>	<code>(7,)</code>	tuple
<code>f()</code>	<code>()</code>	tuple

Because a tuple supports `len`, iteration, and functions like `sum`, `args` is convenient for building helpers such as an average that safely handles the empty case. The empty tuple is a normal input, so guard divisions by zero when it appears.

PYTHON

```
# 🐼 args har-case TUPLE hai – proof + stats helper
def avg_of(*nums):
    print(f"args type: {type(nums).__name__} | value: {nums}")
    if not nums:
        return 0
    return sum(nums) / len(nums)

print("avg:", avg_of(10, 20, 30))
print("avg:", avg_of(7))
print("avg:", avg_of(), "← empty safe ✅")
# Expected OUTPUT:
# args type: tuple | value: (10, 20, 30)
# avg: 20.0
# args type: tuple | value: (7,)
# avg: 7.0
# args type: tuple | value: ()
# avg: 0 ← empty safe ✅
```

4. print() — The *args Superstar

`print` accepts any number of items, which is exactly `*args` in action: `print(a, b, c)`. Its `sep` and `end` options are keyword arguments, so the full signature is roughly `print(*objects, sep=' ', end='\n')`.

A useful pattern is **forwarding**: define `def wrapper(*args): return print('FORWARDED:', *args)`. The star collects the extra positionals, and a second star at the call site unpacks them into `print`. This is how decorators and thin wrappers pass arguments through unchanged while adding behaviour.

Remember the star is symmetric: at a definition it packs, and at a call site it unpacks.

PYTHON

```
# 🐼 print() khud *args ka GOAT 🐐 – siphai example!
print("print", "takes", "unlimited", "items", "(sep=' ' default)")
print("a", "b", "c", sep=" - ", end="!\n") # sep/end = print ke kwargs!

def private_forward(*args):
    return print("FORWARDED:", *args) # args agle pe pass – star uthta!
private_forward(1, 2, 3)
# Expected OUTPUT:
# print takes unlimited items (sep=' ' default)
# a - b - c!
# FORWARDED: 1 2 3
```

5. Signature Anatomy — The Order Laws

Python fixes the order of parameter kinds. A well-formed signature reads:

Call-side rules mirror this: positionals first, then keywords. Definition-side rules: required parameters precede defaults, `*args` precedes keyword-only parameters, and `**kwargs` comes last.

Anything placed after the bare `*`, or after `*args`, can only be passed by keyword. This creates a natural strict zone for optional flags. Used well, keyword-only parameters make intent explicit and make signatures safe to extend later.

PYTHON

```
def mega(pos1, pos2,          # 1) required positionals
        name='default',      # 2) parameters with defaults
        *args,               # 3) extra positionals, packed as a tuple
        kw_only=0,           # 4) keyword-only parameters (after the star)
        **kwargs):           # 5) extra keywords, packed as a dict
    ...
```

5.1 Keyword-Only Arguments

A bare `*` in a signature means everything after this must be passed by name. In `def make_bill(item, price, *, discount=0, delivery=0)`, the discount and delivery cannot be supplied positionally.

This is deliberate API design. Keyword-only flags are self-documenting at the call site (`discount=0.1` reads better than a mystery positional), impossible to confuse by order, and forward-compatible: adding a new keyword-only option never shifts the meaning of existing positional calls.

Passing such an argument positionally raises `TypeError`. The experiment below shows the error message, which confirms the strict zone.

PYTHON

```
# 🚩 Keyword-ONLY demo - `` ke baad wale strict-honest!
def make_bill(item, price, *, discount=0, delivery=0):
    return price - price * discount + delivery

print(make_bill("Laptop", 48000))
print(make_bill("Laptop", 48000, discount=0.1, delivery=500))
try:
    make_bill("Laptop", 48000, 0.1)          # kw-only ko positional se nahi de sakte!
except TypeError as e:
    print("kw-only positional 🚩:", e)
# Expected OUTPUT:
# 48000.0
# 43700.0
# kw-only positional 🚩: make_bill() takes 2 positional arguments but 3 were given
```

Unpacking & **kwargs — Session 03: Named Extras and Star Openers

1. **kwargs — The Tagged Gift-Bag

Placing `**kwargs` in a signature collects any extra *named* arguments into a dictionary called `kwargs`. The double star is the keyword counterpart of `*args`: one packs positionals into a tuple, the other packs keywords into a dict.

Because the result is an ordinary dictionary, use `.get(key, default)` to read optional fields gracefully — a missing key returns the default instead of raising `KeyError`. An empty call is perfectly valid and yields an empty dictionary.

The bag carries *tags*, so order never matters; each value travels with its name. That is what makes `**kwargs` ideal for optional configuration.

PYTHON

```
# 🎁 **kwargs – named extras ka dict-pack! 🎁
def profile(**info):
    print(f"type: {type(info).__name__} | keys: {sorted(info)}")
    print(f"  name: {info.get('name', 'N/A')}")
    print(f"  city: {info.get('city', 'unknown')}")

profile(name="Sana", city="Delhi", hobby="chess")
print()
profile() # empty-gift-bag OK!
# Expected OUTPUT:
# type: dict | keys: ['city', 'hobby', 'name']
#   name: Sana
#   city: Delhi
#
# type: dict | keys: []
#   name: N/A
#   city: unknown
```

PYTHON

```
def profile(**info):
    ...
profile(name='Sana', city='Delhi', hobby='chess')
# info -> {'name': 'Sana', 'city': 'Delhi', 'hobby': 'chess'}
```

2. The Full Signature

A single signature can combine every parameter kind: `def show(order_id, *items, urgent=False, **meta)`. Reading it left to right, `order_id` is a required positional, `items` is a tuple of any extra positionals, `urgent` is keyword-only, and `meta` absorbs all remaining named arguments.

At the call `show(101, 'book', 'pen', urgent=True, gift='yes')`, the pieces fall into place: `order_id=101`, `items=('book', 'pen')`, `urgent=True`, and `meta={'gift': 'yes'}`. The order laws from the theory notebook guarantee an unambiguous split.

This pattern is the backbone of flexible APIs: a small set of required values, a variadic payload, a few explicit flags, and an open bag for the rest.

PYTHON

```
# 🌟 FULL-SIGNATURE combo – sab ek saath! (pos → *args → kw-only → **kwargs)
def show(order_id, *items, urgent=False, **meta):
    print(f"order#{order_id} items={items}")
    print(f"    urgent={urgent} | meta={meta}")

show(101, "book", "pen", urgent=True, gift="yes", note="call-back")
# Expected OUTPUT:
# order#101 items=('book', 'pen')
#    urgent=True | meta={'gift': 'yes', 'note': 'call-back'}
```

3. Call-Side Unpacking

The star works in both directions. At a call site, `*iterable` spreads a sequence into positional arguments and `**mapping` spreads a dictionary into keyword arguments.

- `add3(*[10, 20, 30])` is equivalent to `add3(10, 20, 30)`.
- `welcome(**{'name': 'Ravi', 'city': 'Pune'})` matches arguments by name, so dictionary order is irrelevant.

This is how stored data becomes a function call. `f(*list)` requires the list length to match the parameters, while `f(**dict)` requires the keys to name existing parameters, or a `**kwargs` slot to catch the rest. Mismatches raise `TypeError`.

PYTHON

```
# 🌟 Call-side UNPACKING – star se dabba khoLo 🌟 (List → args!)
def add3(a, b, c):
    return a + b + c

nums = [10, 20, 30]
print("add3(*nums)      :", add3(*nums), "← list ko * se KHOLA – 3 args ban!")

def welcome(name, city):
    print(f"Welcome {name} from {city}!")

student = {"city": "Pune", "name": "Ravi"} # order different – dict matches BY NAME!
welcome(**student) # ** se named-arg bomb!
# Expected OUTPUT:
# add3(*nums)      : 60 ← list ko * se KHOLA – 3 args ban!
# Welcome Ravi from Pune!
```

4. Counting the Bags

A helper can accept both packs at once: `def packer(*args, **kwargs)`. Inside, `len(args)` counts the anonymous values and `len(kwargs)` counts the tagged ones. Calling it with nothing returns `anonymous=0 tagged=0`, which is a valid and common case.

This is a useful debugging and proxy pattern: a wrapper can report or log how many arguments it received before forwarding them onward. Because both packs are ordinary containers, inspecting them is easy.

PYTHON

```
# 🍛 kwargs-length check + kw-detective helpers
def packer(*args, **kwargs):
    return f"anonymous={len(args)} tagged={len(kwargs)}"

print(packer(1, 2, 3, x=1, y=2))
print(packer()) # zero-flexibility legit!
# Expected OUTPUT:
# anonymous=3 tagged=2
# anonymous=0 tagged=0
```

5. functools.partial — Freezing Arguments

`functools.partial` creates a new callable with some arguments already fixed. `eighteen_off = partial(discount_calc, 0.18)` freezes the discount rate, leaving a function that takes only a price. Calling `eighteen_off(48000)` gives the same result as `discount_calc(0.18, 48000)`.

This is a light form of currying: pre-supply the parts that rarely change, and expose a simpler callable for the parts that do. It is handy for callbacks, mapping a transformation over data, and building configured helpers without writing a wrapper function.

PYTHON

```
# 🍛 partial teaser – curry-ka-chhota-bhai 🍛 (Section 04 OOP ke side-serves!)
from functools import partial

def discount_calc(rate, price):
    return round(price * (1 - rate), 2)

eighteen_off = partial(discount_calc, 0.18) # rate FIXED, price open!
print("partial      :", eighteen_off(48000), "← Laptop ka 18%-off price")
print("original fnc :", discount_calc(0.18, 48000), "(same result – freeze-then-use convenience!)")
# Expected OUTPUT:
# partial      : 39360.0 ← Laptop ka 18%-off price
# original fnc : 39360.0 (same result – freeze-then-use convenience!)
```

6. Real Dataset — The Discount Lab

This section loads `products_catalog.csv` into a name-to-price dictionary and applies a variadic discount function. `discounted_list(rate, *prices)` accepts a rate followed by any number of prices and returns the discounted values.

Calling it as `discounted_list(0.18, *first3)` unpacks the first three prices into positional arguments, demonstrating definition-side packing and call-side unpacking together. Zipping the names with the results produces a small, readable report — the pattern used throughout data tooling.

SQL

```
# 📁 REAL DATASET 📁 Products ka Flexible Discount-Lab! 🏷️
import csv

def load_products():
    for path in ["../00_Resources/datasets/products_catalog.csv",
                "00_Python/00_Resources/datasets/products_catalog.csv"]:
        try:
            with open(path, newline="", encoding="utf-8") as f:
                return list(csv.DictReader(f))
        except FileNotFoundError:
            continue
    raise FileNotFoundError("products_catalog.csv nahi mila!")

products = load_products()
name2price = {p["product_name"]: float(p["price"]) for p in products}
print("catalog:", len(name2price), "products ready 📦")

def discounted_list(rate, *prices):
    return [round(p * (1 - rate), 2) for p in prices]

first3 = list(name2price.values())[:3]
print("\n18%-off first-3 :", discounted_list(0.18, *first3))
print("names zipped :", list(zip(list(name2price)[:3], discounted_list(0.18, *first3))))
# Expected OUTPUT:
# catalog: 30 products ready 📦
#
# 18%-off first-3 : [39360.0, 12300.0, 1640.0]
# names zipped : [('Laptop', 39360.0), ('Smartphone', 12300.0), ('Headphones', 1640.0)]
```

7. The Receipt Printer

`receipt(customer, *items, discount=0.0, gst=0.0, delivery=0.0, **flags)` is the full signature in a realistic setting. It sums item prices from the catalogue, applies discount, then GST, then delivery, and finally prints the remaining unknown flags.

Every parameter kind does a job: `customer` is required, `items` is variadic, the money values are keyword-only with defaults, and `flags` absorbs options such as `gift_wrap='yes'`. The example bill shows the arithmetic step by step, and the flags line proves the open bag is still available for future options.

PYTHON

```
# 📄 Receipt printer — **kwargs style optional-fields ka showcase! 📄
def receipt(customer, *items, discount=0.0, gst=0.0, delivery=0.0, **flags):
    subtotal = sum(name2price[i] for i in items)
    after_disc = subtotal * (1 - discount)
    after_gst = after_disc * (1 + gst)
    final = after_gst + delivery
    print(f" 📄 {customer}'s cart: {len(items)} items")
    for i in items:
        print(f"    - {i}: ₹{name2price[i]:,.0f}")
    print(f"    subtotal ₹{subtotal:,.0f} | -{int(discount*100)}% → ₹{after_disc:,.0f}")
    print(f"    +{int(gst*100)}% GST → ₹{after_gst:,.0f} | +delivery ₹{delivery:,.0f}")
    print(f"    FINAL: ₹{final:,.0f} | flags: {sorted(flags)}")

receipt("Rahul", "Laptop", "Smartphone", "Headphones",
       discount=0.10, gst=0.05, delivery=500, gift_wrap="yes", festival="diwali")
# Expected OUTPUT:
# 📄 Rahul's cart: 3 items
#    - Laptop: ₹48,000
#    - Smartphone: ₹15,000
#    - Headphones: ₹2,000
#    subtotal ₹65,000 | -10% → ₹58,500
#    +5% GST → ₹61,425 | +delivery ₹500
#    FINAL: ₹61,925 | flags: ['festival', 'gift_wrap']
```

Common Mistakes and Pitfalls

5 COMMON MISTAKES (ZIP/ENUMERATE SPECIAL)

#	❌ MISTAKE	✅ CORRECT
1	Iterating <code>zip</code> again (silent-empty!)	Fresh <code>zip(...)</code> each pass — regen is cheap!
2	<code>enumerate(items, 1)</code> — forgetting the second arg's position	Put faith in the <code>start=1</code> keyword! ✔️ both work, keyword is readable
3	Assuming "all pairs formed" on an uneven zip	Shortest-stops rule! Need to save data? <code>strict=True</code> or <code>zip_longest</code>
4	<code>for i in range(len(lst))</code> in Python	<code>for i, x in enumerate(lst)</code> — Pythonic!
5	Confusion about the sort key of <code>sorted(zip(a,b))</code>	<code>sorted(zip(totals, names))</code> sorts the FIRST element (totals!) — the tuple-first rule

🔧 **Tip:** in `sorted(zip(totals, names), reverse=True)` the tuple's first item decides the order — ranks come from totals, ties break by names (A-Z). Rock-solid trick! 💪

5 COMMON MISTAKES (MAP/FILTER/REDUCE SPECIAL)

#	✗ MISTAKE	✓ CORRECT
1	<code>reduce</code> without an import — <code>NameError</code> ✨	<code>from functools import reduce</code> FIRST
2	Indexing/slicing <code>filter(...)</code> directly	Wrap it in a list: <code>list(filter(...))</code> — a lazy machine!
3	<code>reduce</code> on an empty list (TypeError!)	<code>reduce(fn, [], initial)</code> — initial as a safety cushion
4	Consuming map/filter again (silent-empty!) — iterator habits!	wrap in <code>list()</code> once to store, or regenerate fresh
5	<code>reduce</code> where sum already exists	Built-in GO: <code>sum/max/min/math.prod</code> — use reduce only for custom combines

🎯 **3-second verdict:** is a ready-made function there? → map. Would you write a lambda? → comprehension. One final single value? → builtin or reduce. ✓

5 COMMON MISTAKES (COPY SPECIAL)

#	✗ MISTAKE	✓ CORRECT
1	Thinking <code>b = a</code> is a copy	Aliasing! Copy = <code>.copy()</code> / <code>[:]</code> / <code>copy.copy()</code>
2	Assuming shallow = "fair-use" on nested	The inner parts are shared! Deep copy <code>copy.deepcopy</code>
3	Relying on <code>copy.copy()</code> for nested	It is still shallow — the module or the method, it only copies the outside
4	Forgetting deepcopy for a tuple-of-lists	Even though a tuple is immutable, the inner lists are shared (live in nb2!)
5	Passing a list into a function, mutating it, and expecting ORIGINAL-safe	Even when passed, it is still a reference — a direct effect on the caller! (03.04 details)

🎯 **Rule:** FLAT (no inner mutables) → shallow is OK. NESTED (lists inside, dicts inside...) → deepcopy. Any doubt? Verify LIVE with `id()`! 🐍

5 COMMON MISTAKES (DIAGRAM SPECIAL)

#	❌ MISTAKE	✅ CORRECT
1	Assuming "nested safe" on Dict <code>.copy()</code>	Outer-fresh only! Deep copy for the inner 💖
2	Thinking a tuple slice "made a copy"	Copy of immutables = the same object (optimization!)
3	JSON hack without type-checks	set ❌, tuple→list 🖋️, int-keys→str
4	Ignoring the deepcopy cost (big trees are slow!)	Deepcopy only the needed part; shallow on flat
5	Forgetting the <code>id()</code> check in trial runs	The id mirror → the truth! (build the diagram in your mind)

🎯 **Compass-meter:** flat→shallow(🔪 fast), nested-mutate→deep(🛡️ safe), on-debug→id()
 experiment! — memory never lies 🧠

5 COMMON MISTAKES (FAMILY SPECIAL)

#	❌ MISTAKE	✅ CORRECT
1	<code>x is y</code> for value-comparisons	<code>x == y</code> use <code>is</code> only for identity/None-checks
2	Expecting in-place on <code>+=</code> for every type	list-in-place 🔥, tuple/num/str REBUILD 🧱
3	A list/tuple-with-list in a dict-key	Only immutables (str/num/pure-tuple/frozenset)
4	Business-logic on interning (<code>is</code>)	A CPython implementation-detail — always compare with <code>==</code>
5	"Tuple immutable, so safe" in nested cases	A tuple-with-a-mutable-list = mutability leakage! 🕳️

🎯 **Feeling:** `id()` = the object's aadhaar-card. In mutables the aadhaar stays the same, only the inner-city changes; in immutables a change = a fresh aadhaar! 🇮🇳

5 COMMON MISTAKES (BEHAVIOR SPECIAL)

#	✗ MISTAKE	✓ CORRECT
1	<code>def f(box=[])</code> with a mutable default	<code>def f(box=None)</code> + the None-guard 🙌 ✓
2	Guessing caller-change vs local-only inside a function	In-place=caller-change 🔥; reassign=local-only 🧊; want safety? <code>.copy()</code> 🛡️
3	<code>+=</code> in a string-building loop	<code>"".join(parts)</code> ★
4	Not using the immutable advantage	dict-keys/thread-share/predictability — exploit the 3 powers! 👤
5	Passing an object and trusting OG-safe	Functions share references — design accordingly!

🎯 **Iron rule:** keep default args immutable; if functions poke mutables in-place, document the call-out; build big strings with join! 🧱

5 COMMON MISTAKES (ARGS SPECIAL)

#	✗ MISTAKE	✓ CORRECT
1	<code>f(x=1, 2)</code> positional after keyword	positional first, keyword after! <code>f(2, x=1)</code>
2	<code>def f(a=1, b)</code> default-then-required	required first: <code>def f(b, a=1)</code>
3	Expecting <code>*args</code> to be a list (trying to mutate!)	args is an immutable TUPLE — need a list? convert
4	A positional call for kw-only	name=value is required post- <code>*</code> 📄
5	Mutable defaults! <code>(= [])</code> flashback 🕒	<code>= None</code> + guard (remember 03.04!)

🎯 **Design tip:** use kw-only (`*, flag=...`) in public APIs — callers stay EXPLICIT, fewer bugs; future-proof signatures! ✨

5 COMMON MISTAKES (UNPACKING SPECIAL)

#	❌ MISTAKE	✅ CORRECT
1	A short arg-count on <code>f(*list)</code> (TypeError!)	The function's params = match the list-length, or have a defaults backup
2	A classic function rejecting <code>**dict</code> 's extra-keys — unexpected kwarg TypeError	put a <code>**kwargs</code> gobble-up inside the function
3	Assuming kwargs is the caller's own data	kwargs is Python's FRESH dict — the caller's vars are safe, play freely inside!
4	<code>f(a=1, *args, a=2)</code> duplicate kwarg — TypeError! <code>got multiple values</code>	dedupe the name-collisions
5	<code>**kwargs</code> before <code>*args</code> in the signature	ORDER law: pos → *args → kw-only → **kwargs (fixed!)

 **Library-design rule:** keep a `**kwargs` gobble-slot in public functions — tomorrow's new features get absorbed as keywords, old callers stay unbroken! 🧠 (a pro move)

Edge Cases and Boundaries

4. THE TUPLE-OF-LISTS EDGE CASE

Tuples are immutable, so Python is free to reuse them rather than copy. Slicing a tuple (`mixed[:]`) returns the **same object** — `s is mixed` is `True` — because there is no way to mutate a tuple, so a copy would be pointless.

But a tuple can *contain* a mutable list. `copy.deepcopy(mixed)` then has an interesting job: the tuple itself may be reused while the list inside is cloned to a fresh object. The inner ids differ, proving the goods were duplicated even though the shell was shared.

The lesson: immutability is shallow. A tuple protects only its own slots, not the objects those slots point to. A nested mutable inside an immutable container is still mutable.

Real-World and AI Engineering Relevance

Where this shows up in AI engineering

- iterator and generator protocol
- functional builtins and lazy evaluation

- shallow vs deep copy
- mutability, immutability and identity
- argument binding with *args and **kwargs

EXAM AUR INTERVIEW PERSPECTIVE

Agar koi concept interview ya exam mein poochha jaye, to definition ke saath apna ek chhota practical example zaroor bolo. Sirf theory batane se confidence nahi dikhta - implementation dikhane se dikhta hai.

Quick Reference

Short, representative snippets collected from the unit's notebooks. Use them as syntax reminders, not as complete programs.

PYTHON**2. YOUR FIRST GENERATOR**

```
def gen_demo():  
    yield "first statement"  
    yield "second statement"  
    yield "third statement"
```

PYTHON**2. YOUR FIRST GENERATOR**

```
gen = gen_demo()  
  
for i in gen:  
    print(i)
```

PYTHON**2.3 GENERATORS REMEMBER WHERE THEY STOPPED**

```
def square(num):  
    for i in range(1,num+1):  
        yield i**2
```

PYTHON

2.3 GENERATORS REMEMBER WHERE THEY STOPPED

```
gen = square(10)

print(next(gen))
print(next(gen))
print(next(gen))

for i in gen:
    print(i)
```

PYTHON

2.3 GENERATORS REMEMBER WHERE THEY STOPPED

```
print(next(gen)) # 1
print(next(gen)) # 4
print(next(gen)) # 9
```

PYTHON

3. `RANGE()` AGAIN — NOW IN FOUR LINES

```
def mera_range(start,end):
    for i in range(start,end):
        yield i
```

PYTHON

3. `RANGE()` AGAIN — NOW IN FOUR LINES

```
for i in mera_range(15,26):
    print(i)
```

Glossary

generator

a special kind of iterator you get almost for free by writing a normal function and using the `yield` keyword instead of `return`.

generator function

any function containing `yield`; calling it returns a **generator object**, which is a full iterator.

iterable

any object whose `__iter__` method returns an iterator — lists, strings, dicts, sets, `range` objects and files all qualify.

iterator

an object with both `__iter__` (returning itself) and `__next__` (returning the next value or raising `StopIteration`).

Iteration

simply the act of consuming an iterator until it is exhausted.

scope

the region where a name is directly visible; a **closure** is a function plus the enclosing variables it remembers; a **decorator** is a callable that takes a function and returns a (usually enhanced) function.

decorator

a function that takes a function as input, wraps it with extra behaviour, and returns a new function.

set

not JSON serialisable, so `json`.

keys

converted to strings, so `{42: 'answer'}` becomes `{'42': 'answer'}`.

hashable

`str`, `int`, `float`, `bool`, `tuple` (only if every element is

not hashable

`list`, `dict`, `set`, `bytearray`.

Revision Checklist

Work through this list without looking at the notes. Anything you cannot explain aloud belongs back in the revision queue.



1. The boilerplate generators replace



1.2 The why: values on demand

☐ 2. Your first generator☐ 2.2 `yield` versus `return`☐ 2.3 Generators remember where they stopped☐ 3. `range()` again — now in four lines☐ 4. Generator expressions☐ 5. A practical generator: reading images lazily☐ 6. Benefits of using a generator☐ 6.1 Ease of implementation☐ 6.2 Memory efficient☐ 6.3 Representing infinite streams☐ 6.4 Chaining generators☐ Summary☐ 1. Iteration, Iterables and Iterators☐ What is Iterator☐ What is Iterable☐ Point to remember — the two rules☐ Trick — recognise them with `dir()`☐ 2. How a `for` loop really works



3. Writing our own `for` loop



4. `iter()` on an iterator returns itself